

SPEC CPU2026: Characterization, Representativeness, and Cross-Suite Comparison

Ruihao Li Andrew Jacob Neeraja J. Yadwadkar Lizy K. John
The University of Texas at Austin

Abstract

Specialized accelerators dominate AI workloads, but CPUs remain critical for latency-sensitive workloads, agentic AI, and many other everyday services. Their performance therefore shapes end-to-end system efficiency, raising the question of whether the latest *SPEC CPU* benchmarks change the architectural conclusions drawn from prior generations. We comprehensively characterize *SPEC CPU2026* across nine recent Intel, AMD, Ampere, and Nvidia platforms to understand how the new suite differs from its predecessors and what new capabilities it introduces. We observe that compared with *SPEC CPU2017*, *SPEC CPU2026* increases instruction volume, memory footprint, and instruction-cache stress.

We also compare *SPEC CPU2026* with *SPEC CPU2017*, recent datacenter and machine learning suites DCPerf and MLPerf, and agentic AI probes, using microarchitectural metrics to understand how specialized suites resemble and differ from *SPEC CPU* suites. We note that *SPEC CPU2026* remains a complementary general-purpose suite: closer to datacenter-like frontend pressure than prior *CPU* benchmark generations, yet less vector-intensive than MLPerf and less frontend-extreme than DCPerf. Importantly, the expanded frontend envelope closely matches emerging CPU-centric agentic AI workloads: an agentic pipeline’s features fall inside *SPEC CPU2026*’s behavioral spread, making the suite a ready-made evaluation proxy for this fast-growing workload class. Furthermore, case studies on page sizes and memory allocators, prefetching, compilers, ISA sensitivity, many-core scaling, and rolling round-robin (RRR) runs (new in *SPEC CPU2026*) demonstrate the suite’s utility beyond aggregate scores. Finally, using cluster-based representativeness analysis, we identify subsets of 4-5 workloads per group that preserve 96.4-99.9% of full-suite behavior. Overall, *SPEC CPU2026* updates the standardized general-purpose CPU baseline for the next decade of architecture evaluation.

1 Introduction

General-purpose CPUs remain indispensable: they run desktop and datacenter services [41, 65, 90, 129, 156], coordinate heterogeneous accelerators [60, 87, 138, 149, 158], execute latency-sensitive control and systems code [6, 99, 147], and support workloads that map poorly to specialized hardware [58, 88, 144, 150]. Although the research community has focused on machine learning (ML) models [84, 112, 113, 137], and GPUs [3, 25, 71, 100, 111, 118, 155], FPGAs [2, 22, 36, 120, 151], and custom domain-specific accelerators [24, 26, 35, 54, 55, 61, 62, 146], CPU benchmarking is not a “solved” problem or less relevant in an accelerator-dominated era. Industry reports point to renewed CPU demand from agentic AI deployment [8, 63, 110]. With many emerging trends, rigorous CPU evaluation is increasingly important for the computer architecture community.

SPEC CPU has long been the de facto standard for CPU performance evaluation in both industry and academia [18, 49, 51, 52, 76, 92, 97, 103, 104, 116, 122, 127, 128, 148]. There are more than 40,000 score reports on *SPEC CPU17* and about 300 on *SPEC CPU26* in the three months since release [95]. But ML, cloud/datacenter, and agentic AI workloads [50, 67, 142, 161] have become mainstream, and suites such as MLPerf [112] and DCPerf [134] have emerged. This raises a timely question: what does the newest *SPEC CPU* release – *SPEC CPU2026* (*SPEC CPU26*) – contribute beyond *SPEC CPU17*, and how should it be interpreted relative to modern domain-focused suites?

In this paper, we comprehensively characterize *SPEC CPU26*, compare it to other suites, and draw implications for modern CPU architecture evaluation. We answer four questions that matter to the computer architecture community:

- How does *SPEC CPU26* differ from *SPEC CPU17* in microarchitectural behavior and coverage?
- How does it compare with datacenter suite DCPerf and machine learning benchmark MLPerf? Does its coverage extend to agentic AI? Can adding a dense matrix-multiply kernel based benchmark reduce its distance to MLPerf?
- What other evaluations can *SPEC CPU26* support beyond score reporting?
- Can compact subsets preserve *SPEC CPU26*’s behavioral diversity at lower evaluation cost?

Using microarchitectural characterization and cross-generation similarity analysis, we show that *SPEC CPU26* updates the general-purpose CPU evaluation baseline: it provides meaningful new stress coverage over *SPEC CPU17* while remaining complementary to DCPerf and MLPerf; and its scope already covers agentic AI orchestration. MLPerf is more dissimilar to *SPEC CPU26* than DCPerf, and a dense matrix-multiply kernel narrows this distance for MLPerf’s dense kernels. *SPEC CPU26* shifts stress toward larger instruction footprints, larger working sets, and higher many-core scalability, while reducing dependence on several extreme *SPEC CPU17* outliers. Broader compiler and ISA sensitivity is also observed. These changes matter because modern CPU research increasingly studies high-core-count scaling [131, 133], chiplet and non-chiplet organization [1, 68, 132], heterogeneous memory [126, 135], service co-location [23, 81], compiler/runtime configuration [14, 154], and agentic AI [8, 63, 110].

In summary, this paper makes the following contributions:

- We provide the first detailed characterization of *SPEC CPU26* across nine modern platforms, showing how it differs from *SPEC CPU17* in all four suite groups (§ 2).
- We compare *SPEC CPU26* with *SPEC CPU17*, DCPerf, and MLPerf (§ 3): *SPEC CPU26* narrows the gap to datacenter

Table 1: Hardware configurations of the nine machines used in the experiments (machines are either dual/single-socket systems, we report per-socket configurations in this table).

Machines	CPU-A	CPU-B	CPU-C	CPU-D	CPU-E	CPU-F	CPU-G	CPU-H	CPU-I
Vendor & Model	Intel Skylake	Intel Icelake	Intel Sapphire Rapids	Intel Sapphire Rapids	AMD Milan	AMD Genoa	AMD Turin	Ampere Altra	Nvidia Grace
Release Year	2017	2021	2023	2023	2021	2022	2024	2020	2023
Core	Platinum 8160	Platinum 8380	Platinum 8468	MAX 9480	EPYC 7763	EPYC 9454	EPYC 9555	Neoverse-N1	Neoverse-V2
# of Cores	24	40	48	56	64	48	64	80	72
L1-D/I cache	32/32KB	48/32KB	48/32KB	48/32KB	32/32KB	32/32KB	48/32KB	64/64KB	64/64KB
L2-cache	1MB	1.25MB	2MB	2MB	512KB	1MB	1MB	1MB	1MB
L3-cache/core	1.38MB	1.5MB	2.19 MB	2MB	4MB	4MB	4MB	0.4MB	1.63MB
DIMM	6 × 16 GB	8 × 16 GB	8 × 96 GB	4 × 16 GB	8 × 16 GB	12 × 16 GB	12 × 32 GB	8 × 16 GB	224GB 32-Channel
	DDR4	DDR4	DDR5	HBM2e	DDR4	DDR5	DDR5	DDR4	LPDDR5X
	2666 MT/s	3200 MT/s	4800 MT/s	3200 MT/s	3200 MT/s	4800 MT/s	6000 MT/s	3200 MT/s	8533 MT/s

workloads through greater frontend pressure while remaining distinct from extreme service and ML inference behaviors; it already captures agentic AI orchestration, and a dense matrix-multiply kernel mitigates the dissimilarity to MLPerf’s dense kernels.

- We demonstrate the use cases of *SPEC CPU26* for circa 2026 CPU research (§ 4): compared to *SPEC CPU17*, *SPEC CPU26* sustains higher many-core scaling, is more sensitive to compiler optimizations, shows broader ISA-dependent variation, and enables proxy workload generation via its Rolling Round-Robin (RRR) mode.
- We perform redundancy analysis and representative subset selection (§ 5), showing that 4-5 workloads per group preserve 96.4-99.9% of suite behavior.

Overall, *SPEC CPU26* is a timely suite that updates standardized, comparable CPU evaluation for the next decade.

2 Overview of *SPEC CPU26*

This section summarizes *SPEC CPU26*’s organization, instruction volume, and high-level performance behavior.

2.1 Suite Overview of *SPEC CPU26*

Following *SPEC CPU17*, *SPEC CPU26* is organized into four categories – INT Rate, INT Speed, FP Rate, and FP Speed – with 14, 13, 12, and 13 benchmarks, respectively, totaling 52 benchmarks, 9 more than *SPEC CPU17*. It also retains C, C++, and Fortran. However, all *SPEC CPU26* workloads were developed from scratch rather than inherited from previous suites. Counting Rate and Speed variants as a single workload, only 3 of *SPEC CPU26*’s 19 integer workloads preserve *SPEC CPU17* names (*omnetpp*, *gcc*, and *xz*), and 6 of its 19 floating-point workloads do (*cactus*, *fotonik3d*, *lbm*, *nab*, *namd*, and *roms*). Despite sharing these names, the corresponding *SPEC CPU26* workloads use entirely new codebases and inputs. Recent work [80] describes the development process of *SPEC CPU26*, whereas this paper provides a deeper characterization of the suite across multiple modern processor architectures.

2.2 Performance Characterization

Platforms and metrics. We evaluate *SPEC CPU26* on nine systems (Table 1) spanning x86_64 and AArch64 ISAs, recent Intel, AMD, Ampere, and Nvidia processors, and diverse cache, core-count, and memory configurations. All benchmarks are compiled with gcc -O3, and hardware counters are collected with Linux perf [77]. All

Table 2: Performance metrics used in our analysis.

Category	Perf Metrics
Inst/Cycle	IPC
Cache	L1I\$ MPKI, L1D\$ MPKI, L2\$ MPKI, L3\$ MPKI
TLB	L1 iTLB MPMI, L1 dTLB MPMI, L2 TLB MPMI
Branch Predictor	Branch MPKI
Pipeline	Frontend stall %, Backend stall %
Inst Mix	Kernel%, User%, Load%, Store%, Branch%, FP%, Vector%
DRAM	Mem access (bytes per cycle)

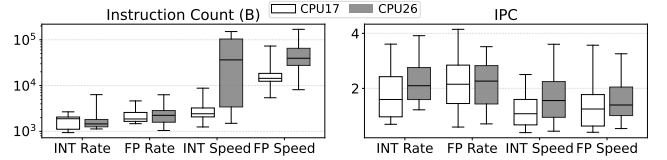


Figure 1: Per-workload dynamic instruction count (log scale) and IPC distributions for *SPEC CPU17* and *SPEC CPU26* across INT and FP, Speed and Rate workloads, pooled over all nine platforms. *SPEC CPU26* increases total instruction count by 23.2× for INT Speed and 3.4× for FP Speed, while Rate footprints grow modestly.

systems use their default deployment configurations, including dynamic frequency scaling, since our analysis relies primarily on cycle- and instruction-normalized metrics. Table 2 lists our used 19 metrics, covering IPC, cache/TLB behavior, branch prediction, frontend/backend stalls, instruction mix, and DRAM access intensity. This feature space follows established architectural characterization practice [65, 97, 104, 129, 134] and captures the major frontend, execution, memory-system, and control-flow bottleneck dimensions of general-purpose CPU workloads.

Figure 1 shows the dynamic instruction count and IPC distributions (per-workload detail in Appendix Table 9) for *SPEC CPU26* and *SPEC CPU17* across the four suite groups, aggregated across the nine platforms. Relative to *SPEC CPU17*, *SPEC CPU26* increases total dynamic instruction count by 1.61×, 1.03×, 23.21×, and 3.38× for INT Rate, FP Rate, INT Speed, and FP Speed, respectively. This growth reflects modern processor scaling, especially higher core counts in Speed runs [4, 13, 32, 91, 94, 98, 123, 125], but also increases detailed simulation cost [15, 44, 66, 97, 102, 121]. Median IPC also rises in every group (e.g., 1.60 → 2.10 for INT Rate).

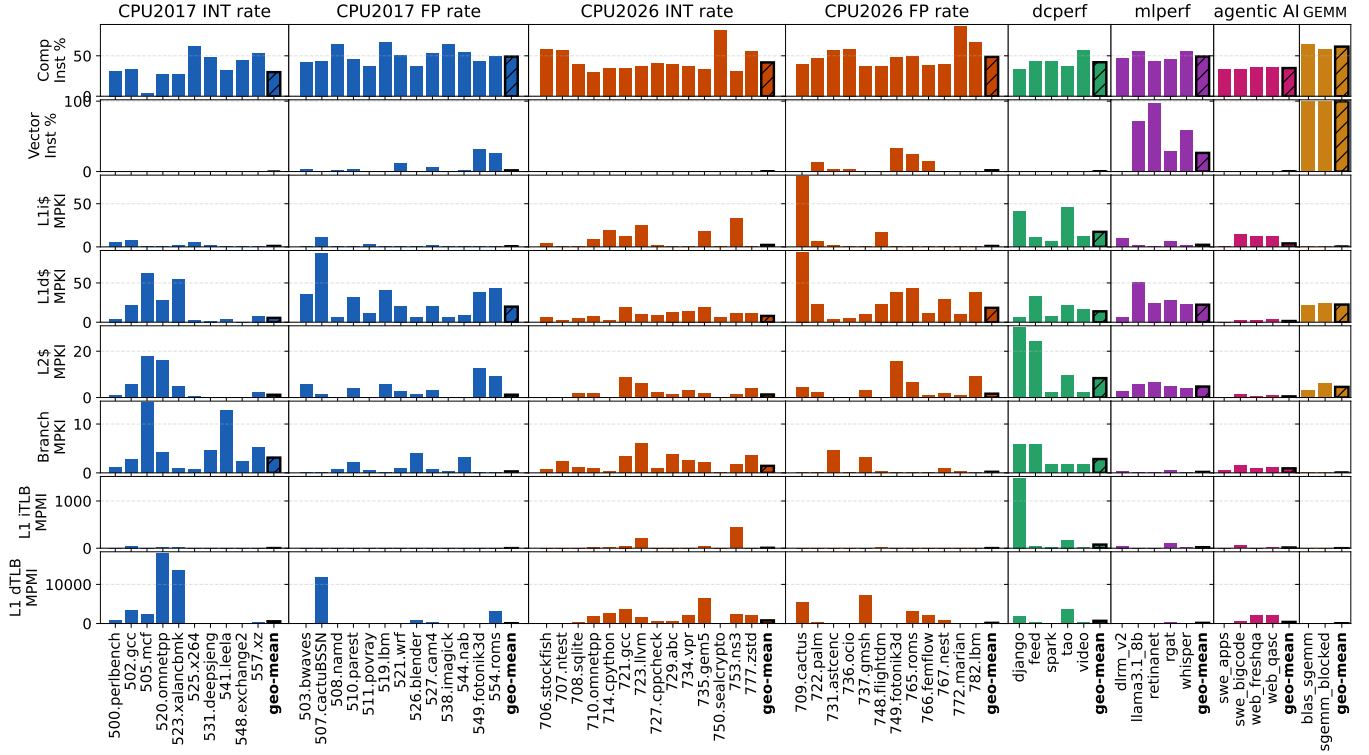


Figure 2: Comparison of key metrics between *SPEC CPU26*, *SPEC CPU17*, MLPerf, DCPerf, an agentic AI suite, and a GEMM kernel; each per-workload value is the median across the nine machines in Table 1. *SPEC CPU26* offers broader coverage than *SPEC CPU17* and the agentic AI suite, while DCPerf places far more stress on the frontend (L1i\$ and iTLB) and MLPerf on vector instructions – with the GEMM kernel further out still, at the vector-throughput extreme.

Takeaway: *SPEC CPU26* substantially increases dynamic instruction volume, especially for Speed workloads, increasing simulation and evaluation cost.

3 *SPEC CPU26* vs. *SPEC CPU17*, DCPerf, MLPerf, Agentic AI, GEMM

This section compares *SPEC CPU26* with its predecessor *SPEC CPU17* and two production-driven suites, DCPerf and MLPerf (Table 3). DCPerf targets hyperscale datacenter services with large instruction footprints and high frontend pressure [134], whereas MLPerf Inference emphasizes ML-centric tensor/vector and floating-point behavior under standardized model, dataset, and accuracy rules [112]. But how similar are these suites to *SPEC CPU26*? Since *SPEC CPU17* primarily consists of standalone general-purpose CPU programs, we explore whether *SPEC CPU26* tilts this general-purpose nature toward modern datacenter-adjacent behavior, while remaining distinct from domain-specific suites such as DCPerf and MLPerf. To probe the edges of that scope, we add two widely used agentic AI pipeline benchmarks [110] (an SWE agent on APPS/BigCodeBench tasks [50, 161] and a web-augmented agent on FreshQA/QASC queries [67, 142]), motivated by the CPU demand [8, 63, 110]. We also a dense fp32 GEMM (8192³), run as both a hand-written cache-blocked kernel [43] and OpenBLAS [153], in some ways similar to matrix300 from *SPEC89*. For the agentic pipeline we profile

Table 3: Benchmarks used in our analysis.

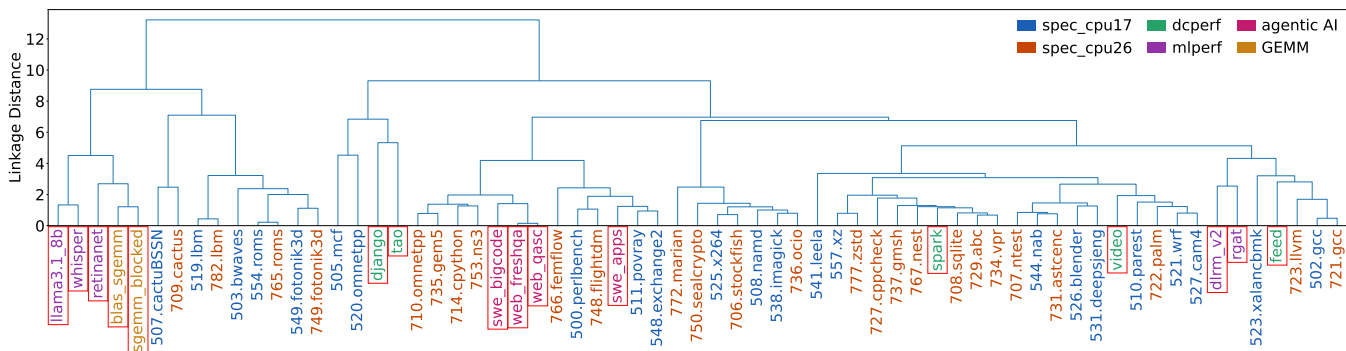
Category	Workloads
<i>SPEC CPU17</i>	Workloads indicated by numbers 5xx in Figure 2
<i>SPEC CPU26</i>	Workloads indicated by numbers 7xx in Figure 2
DCPerf	django, feed, spark, tao, video
MLPerf	dlrm_v2, llama3.1_8b, retinanet, rgat, whisper
Agentic AI	swe_apps, swe_bigcode, web_freshqa, web_qasc
GEMMs	blas_sgemm, sgemm_blocked

the *orchestration* rather than the LLM serving backend (Llama-3.1-8B [46]), since MLPerf’s *llama3.1_8b* already represents LLM inference in this comparison. We analyze detailed microarchitectural metrics (§ 3.1) and workload-level similarity (§ 3.2).

3.1 Detailed Microarchitectural Analysis

We first compare *SPEC CPU26*, *SPEC CPU17*, DCPerf, MLPerf, the agentic AI suite, and GEMM on raw performance metrics/features. Figure 2 summarizes compute intensity, cache and branch behavior, and address-translation pressure for every workload of the six groups; per-workload medians across all nine machines are robust to machine-specific outliers.

Compute and vector intensity: For INT Rate, *SPEC CPU26* has higher fraction of non-load/store/branch instructions over *SPEC CPU17* (41.83% vs. 29.77%, or 1.40×), matching DCPerf (42.04%); FP Rate shows a similar compute-intensity range across all three suites



(~49%). MLPerf, however, remains clearly distinct in vector instruction stress: its vector share reaches 26.48%, versus 1.72% and 1.81% in *SPEC CPU17* and *SPEC CPU26* FP Rate (and effectively zero in both INT Rate suites). The GEMM kernels form the extreme of that same axis, with the highest compute-instruction share (61.36%) and the lowest branch (0.013 MPKI) and instruction-side pressure of any group measured. The agentic AI suite sits at the other end, with effectively no vector work and a compute-instruction share (34.81%) below every group except *SPEC CPU17* INT Rate. As expected, *SPEC CPU26* is not primarily a vector-throughput stress test; its main value lies in frontend, cache, control-flow, and translation behavior studies.

Frontend and translation pressure: The largest differences between *SPEC CPU17* and *SPEC CPU26* appear in instruction-cache and translation behavior. For L1I\$ MPKI, *SPEC CPU26* rises by $2.11\times$ over *SPEC CPU17* in INT Rate and $2.69\times$ in FP Rate, substantially increasing frontend pressure while remaining well below DCPerf (17.41 MPKI). Still, *SPEC CPU26* bridges the DCPerf gap more closely than *SPEC CPU17*: 709.cactus_r reaches L1I\$ stress comparable to some DCPerf workloads, while remaining distinct on service-style dimensions (§ 3.2). L1 iTLB pressure grows faster, from 2.51 to 7.99 MPMI in INT Rate ($3.18\times$) and from 0.99 to 1.98 in FP Rate ($2.00\times$), and L1 dTLB pressure rises by $1.42\times$ in INT Rate and $2.32\times$ in FP Rate. The agentic AI suite lands between *SPEC CPU26* and DCPerf on the instruction side (L1I\$ 4.02 MPKI vs. 2.26 for *SPEC CPU26* INT Rate and 17.41 for DCPerf; L1 iTLB 14.69 MPMI vs. 7.99 and 76.26), while its data-side pressure is the lowest measured (L1D\$ 1.52 MPKI, L2\$ 0.50 MPKI) – a Python/HTTP orchestration profile rather than a datacenter-service one [50, 67, 110, 142, 161]. *SPEC CPU26* thus expands frontend and translation stress relative to *SPEC CPU17*, though DCPerf’s instruction-side stresses (L1I\$ and iTLB) remain far larger.

Cache and branch behavior: Data-cache changes are more moderate. L1D\$ MPKI increases by 1.45 $\times$ in INT Rate but is essentially unchanged in FP Rate (19.83 vs. 18.31), while L2\$ MPKI rises modestly, from 1.20 to 1.32 in INT Rate and from 1.22 to 1.66 in FP Rate. Both SPEC suites exhibit far lower L2\$ pressure than DCPeRf (8.38) and MLPerf (4.75) at the suite level. GEMM, despite its minimal instruction-side footprint, matches MLPerf’s data-cache intensity

(L1D\$ 22.48 vs. 22.29 MPKI; L2\$ 4.60 vs. 4.75): dense blocked math streams data while exercising little else. In contrast, branch pressure decreases: *SPEC CPU26* INT reduces branch MPKI from 3.12 to 1.43 (0.46 $\times$ of *SPEC CPU17*), and FP Rate also decreases slightly.

Takeaway: (1) Compared with *SPEC CPU17*, *SPEC CPU26* broadens general-purpose CPU coverage primarily by increasing instruction-cache, frontend, and translation pressure while substantially reducing branch-miss pressure. (2) *SPEC CPU26* remains complementary to domain-specific suites: *SPEC CPU26* captures some DCPerf-like characteristics without reproducing DCPerf’s extreme hyperscale instruction-cache and translation demands, and remains distinct from MLPerf’s highly parallel inference behavior.

3.2 Similarity Analysis

Similarity analysis methodology. The per-metric comparison treats each dimension independently; we next quantify holistic workload similarity using clustering and principal component analysis [97, 104, 109]. First, we normalize all performance metrics so differences reflect workload behavior rather than metric scale or machine-specific magnitude. Second, we apply Principal Component Analysis (PCA) to the cross-platform feature matrix, where each of the 19 metrics on each of the nine platforms is a distinct feature (171 dimensions before reduction). PCA removes correlated dimensions while preserving dominant behavioral variation [97, 104, 109]. Finally, we apply hierarchical clustering in the PCA-reduced space and use linkage distance to quantify workload similarity and guide representative-subset selection.

Clustering the *SPEC CPU26* Rate, *SPEC CPU17* Rate, DCPerf, MLPerf, agentic AI, and GEMM workload features yields the dendrogram in Figure 3. DCPerf workloads have linkage distances of 1-2 to several *SPEC CPU26* workloads, indicating that *SPEC CPU26* captures some datacenter-relevant microarchitectural properties within its broader general-purpose scope, while DCPerf still represents service-specific extremes. In contrast, MLPerf workloads remain far from both *SPEC CPU26* and *SPEC CPU17* (linkage distance ≥ 3 , up to 8), reflecting a different execution regime dominated by vector-heavy, data-parallel inference kernels. The MLPerf linkage distance is not a coverage failure; it indicates *SPEC CPU26*’s scope as a general-purpose CPU suite rather than a specialized

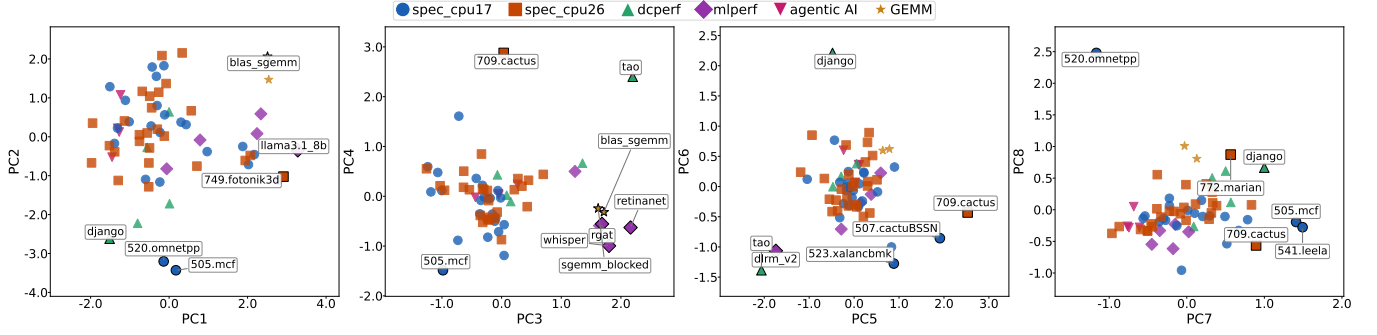


Figure 4: Principal Component (PC) comparison across six benchmark suites. The top 8 PCs capture 80.9% of total variability; outlier and unique workloads plot far from the central cluster. Notably, 505.mcf_r is an outlier in *SPEC CPU17* and is not included in *SPEC CPU26*. DCPerf (tao, django) exhibits unique signatures on PC3/PC4 and PC6; MLPerf workloads are primarily distinguished by PC3, with the GEMM kernels alongside them toward the far right of PC3.

Table 4: Dominant metrics per PC (top 4 by |mean loading|).

PC	Top 4 metrics (value in parens = mean of loadings over machines)
1	BranchIns(-0.12), L3\$_MPKI(0.10), FLOPs(0.10), VectorIns(0.10)
2	ipc(0.15), L2\$_MPKI(-0.11), L3\$_MPKI(-0.09), BranchIns(-0.09)
3	VectorIns(0.12), FLOPs(0.10), Mem_Access(-0.09), KernelIns(0.08)
4	L1\$_MPKI(0.16), BranchIns(-0.14), Branch_MPKI(-0.11), L1D\$_MPKI(0.08)
5	LoadIns(0.14), L1D\$_MPKI(0.14), KernelIns(-0.10), UserIns(0.10)
6	L1_iTLB_MPMI(0.13), L2\$_MPKI(0.10), Frontend(0.09), L1_dTLB_MPMI(-0.09)
7	Branch_MPKI(0.21), StoreIns(-0.11), BranchIns(-0.10), L1_dTLB_MPMI(-0.06)
8	L2_TLB_MPMI(0.16), L1_dTLB_MPMI(0.13), VectorIns(0.08), BranchIns(-0.08)

accelerator-oriented inference suite. The agentic AI workloads fall inside *SPEC CPU26*’s cluster, joining 710.omnetpp_r, 735.gem5_r, 714.cpython_r, and 753.ns3_r at linkage 2.0; its distance to the nearest *SPEC CPU26* workload (0.9-1.6) lies within *SPEC CPU26*’s internal nearest-neighbor spread (median 1.0), so *SPEC CPU26* already covers this behavior. Mixed *CPU17*/*CPU26* clusters reflect overlap in general-purpose behavior, whereas isolated locations indicate domain-specific or outlier characteristics. Overall, *SPEC CPU26* approaches some DCPerf workload behaviors while remaining distinct from the MLPerf suite.

The GEMM kernel probes another boundary: would adding a dense matrix multiply – similar to a matrix benchmark *SPEC89* [29] carried until 1991 – bring *SPEC CPU26* closer to MLPerf? Only partly. The kernel itself is MLPerf-like – in Figure 3 it merges into the dense-inference branch, nearer to *retinanet* (2.38) than to any *SPEC CPU26* workload (3.72) – so including it cuts the suite’s distance to MLPerf’s dense kernels from 3.58 to 3.09. The recommendation and graph workloads (*dllrm_v2*, *rgat*) – already *SPEC CPU26*’s nearest MLPerf neighbors – are untouched, so the augmented suite closes only 9% of the overall gap: a GEMM buys the dense-kernel corner, not MLPerf.

Figure 4 and Table 4 further illustrate the top eight principal components (in our PCA based clustering analysis). Figure 4 shows that *SPEC CPU17* had outlier workloads (505.mcf_r and 520.omnetpp_r), while *SPEC CPU26* has fewer outliers and more stable suite-level behavior. DCPerf programs *tao* and *django* show uniqueness in many principal components. MLPerf is unique primarily along vector and memory-intensity dimensions (PC3), whereas DCPerf separates through service-style frontend behavior (e.g., *tao*

along PC3/PC4 and *django* along PC6) [134]. The GEMM kernels sit near the MLPerf programs toward the far right of PC3, while the agentic workloads plot inside the *SPEC CPU26* cloud.

Takeaway: *SPEC CPU26* contains fewer extreme outliers than *SPEC CPU17*, yielding more stable suite-level behavior. Meanwhile, *SPEC CPU26* moves closer to datacenter-like behavior represented by DCPerf while remaining clearly separated from highly vector-centric MLPerf, defining its modern general-purpose scope. The two probes bound this scope from both sides: agentic AI is largely covered by *SPEC CPU26*’s existing high-footprint workloads, whereas a dense GEMM reaches only MLPerf’s dense-kernel corner and leaves its recommendation and graph workloads as far away as before.

4 Use Cases of *SPEC CPU26*

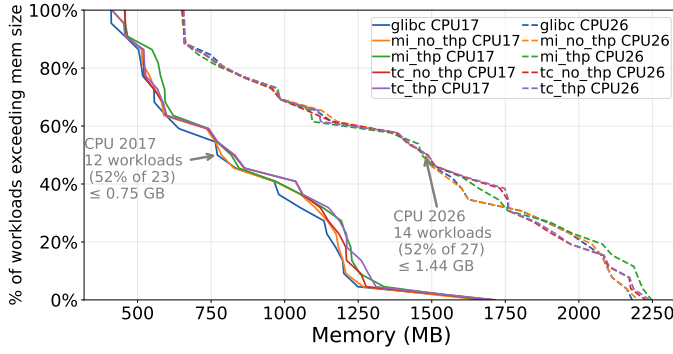
Beyond suite-level characterization, *SPEC CPU26* exposes practical architectural and system-level sensitivities through case studies on system configurations (§ 4.1), architectural choices (§ 4.2), and proxy workload construction (§ 4.3).

4.1 Evaluating System Configurations

We show how *SPEC CPU26* can evaluate key system-configuration choices that improve performance without changing the underlying silicon [129], including memory requirements (§ 4.1.1) and page sizes (§ 4.1.2), hardware prefetcher (§ 4.1.3), and compiler (§ 4.1.4).

4.1.1 Memory Requirements. Working Set Size (WSS) is an important attribute that determines memory usage and system requirements; the concept dates back to Denning’s early work [28]. We use the resident set size (RSS, *rss* in the Unix utility *ps*), which measures how much memory an application actually uses, as a practical proxy for WSS [45] to compare the memory footprints of the Rate workloads. Because memory usage is heavily affected by libraries, allocators, and optimizations, we study three allocators: *glibc*, *tcmalloc*, and *mimalloc* (more details in § 4.1.2). *SPEC CPU26* RSS reaches 2.2 GB, compared to 1.7 GB for *SPEC CPU17*.

Figure 5a shows the results on CPU-C; other platforms show similar trends. *SPEC CPU26* has a larger memory footprint than *SPEC CPU17*, with a median RSS of approximately 1.44 GB versus 0.75 GB. A few *SPEC CPU26* workloads share names with predecessors, but



(a) Complementary CDF of all Rate workloads.



(b) RSS over time for 721.gcc_r.



(c) RSS over time for 709.cactus_r.

Figure 5: Memory Resident Set Size (RSS) for *SPEC CPU26/SPEC CPU17* Rate on CPU-C. (a) *SPEC CPU26* RSS is larger than *SPEC CPU17* (per-workload data in Appendix Table 10 and Table 11). (b) gcc has varying RSS, also heavily dependent on THP. (c) cactus has varying RSS, also heavily dependent on THP.

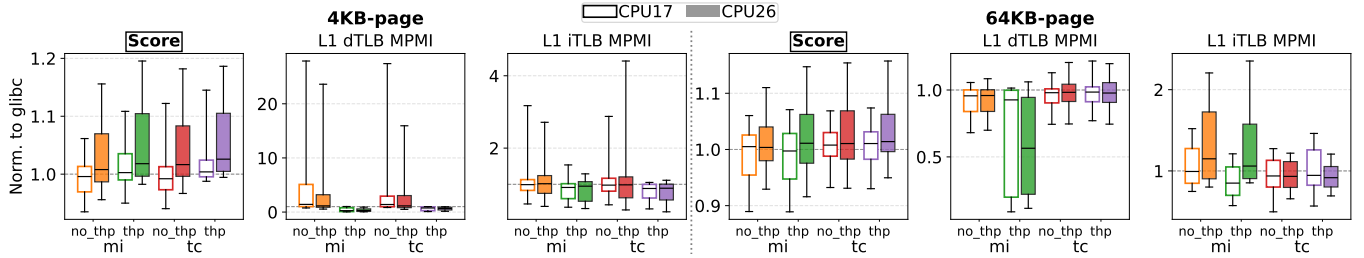


Figure 6: Performance of mimalloc (mi) and tcmalloc (tc), with and without THP, across *SPEC CPU26* and *SPEC CPU17* Rate workloads, normalized to the corresponding glibc baseline (per-workload data in Appendix Figure 15 and Figure 16, the most 2 representative platforms CPU-C and CPU-I). With 4 KB pages, advanced allocators and THP benefit *SPEC CPU26* more than *SPEC CPU17*. With 64 KB pages, both techniques are less effective. The performance gains primarily come from reductions in both L1 dTLB and L1 iTLB misses, with THP producing larger reductions in L1 dTLB misses.

their codebases and inputs are entirely new, so we treat cross-suite comparisons as indicative rather than direct continuations. Within *SPEC CPU26*, six workloads exceed 2.0 GB (Table 5), whereas no *SPEC CPU17* Rate workload reaches that threshold. The shift is therefore driven primarily by newly introduced workloads with natively high memory demands and by enlarged inputs on same-named scientific benchmarks, not by moderate datacenter-oriented additions (708.sqlite_r, 714.cpython_r, 723.llvm_r, 753.ns3_r).

How allocators affect RSS? In general, the choice of allocator does not have a strong effect on the RSS at the suite-level (Figure 5a). Beyond suite averages, temporal RSS traces reveal strong allocator sensitivities (Figure 5b and Figure 5c). For 721.gcc_r, peak RSS differs by up to ~0.4 GB across allocators, while transient variations exceed 1 GB, indicating strong allocator sensitivity beyond the steady-state working-set size (similar difference exists in 709.cactus_r as well). **Takeaway:** The median memory consumption of *SPEC CPU26* (1.44 GB) is almost twice that of *SPEC CPU17* (0.75 GB), which amplifies its sensitivity to page size and translation overheads for memory-intensive workloads.

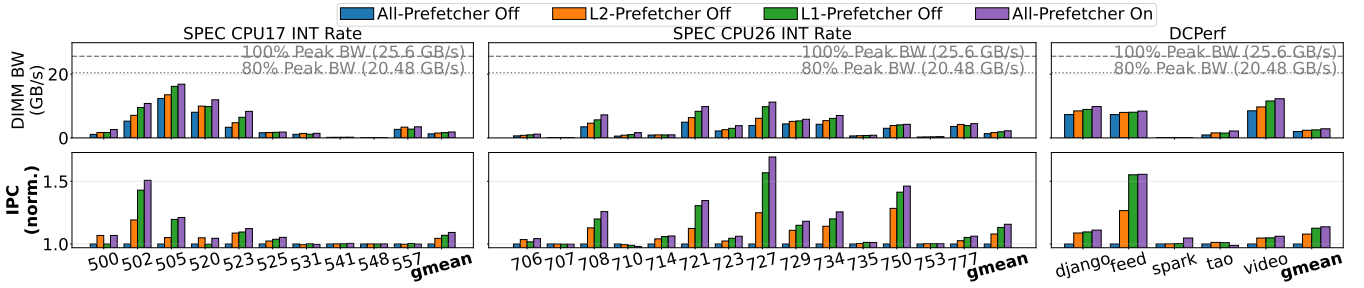
4.1.2 Page Size. Page size is a common system-level tuning knob. Larger pages can reduce translation overhead and page-walk traffic, but may increase fragmentation and alter address translation behavior [9, 20, 27, 48, 74, 159]. To evaluate how *SPEC CPU26* and *SPEC CPU17* respond to page size and transparent huge pages (THP), we

Table 5: Memory resident set size (RSS) groupings of *SPEC CPU26* Rate workloads (max RSS across allocators on CPU-C).

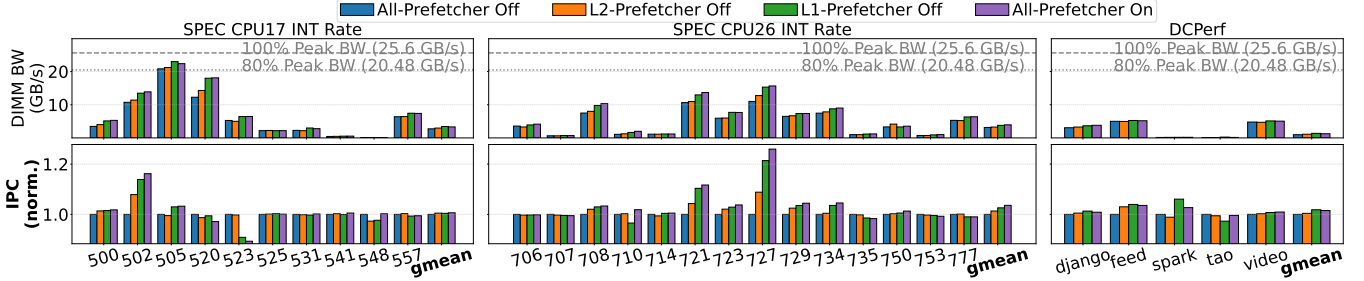
Low RSS (<1 GB)	Medium RSS (1–2 GB)	High RSS (≥2 GB)
707.ntest, 727.cppcheck, 731.astcenc, 748.flightdm, 750.sealcrypto, 753.ns3, 766.femflow, 777.zstd	708.sqlite, 710.omnetpp, 714.cpython, 721.gcc, 723.llvm, 729.abc, 735.gem5, 736.ocio, 737.gmsh, 765.roms, 767.nest, 772.mar-ian	706.stockfish, 709.cactus, 722.palm, 734.vpr, 749.fon- tonik3d, 782.lbm

analyze seven platforms: five x86_64 servers with 4KB base pages and two AArch64 servers with 64KB base pages. We exclude the oldest (CPU-A) and newest (CPU-G) x86_64 systems to better align hardware generations with the AArch64 nodes. The microarchitectural impact of page size can be obscured by allocator behavior, since the default glibc allocator does not consistently support allocations with THPs [160]. To reduce this bias, we use two additional allocators, mimalloc and tcmalloc, which explicitly support and manage huge pages [53, 75]. Figure 6 plots per-workload score distributions for *SPEC CPU26* and *SPEC CPU17* for 4KB and 64KB page sizes.

On 4KB-page systems, exposing proper THP behavior via these allocators reveals noticeable suite-level sensitivity to translation overheads. Relative to the baseline glibc, migrating to THP-aware



(a) Results on CPU-B. Higher IPC comes from better DIMM bandwidth utilization.



(b) Prefetcher efficiency drops on CPU-E compared with CPU-B as more cores (64 vs. 40) share the same DIMM bandwidth.

Figure 7: IPC and DIMM bandwidth across prefetcher configurations on CPU-B and CPU-E. IPC is normalized to the all-prefetchers-off configuration (4th bar in each group shows the overall prefetching benefit). On CPU-B, 6 of 14 *SPEC CPU26* INT Rate workloads improve by at least 10%, and *SPEC CPU26* shows sensitivity comparable to DCPerf (1.15× vs. 1.13×) and greater than *SPEC CPU17* (1.06×). On the bandwidth-limited CPU-E, *SPEC CPU26* remains the most sensitive suite (1.04× vs. 1.02× for DCPerf and 1.01× for *SPEC CPU17*).

allocators improves the geometric-mean *SPEC CPU26* score by 3.1-6.7% and *SPEC CPU17* by 1.1-5.7% (averaged across the 4KB platforms), with individual microarchitectures seeing *SPEC CPU26* gains up to 12.7%. These gains confirm that 4KB-page systems suffer from translation bottlenecks that proper huge-page support can alleviate. Specifically, *tcmmalloc* with THP reduces L1 dTLB and L1 iTLB MPMI by ~52% and ~28%, respectively, on average, reaching up to 65.3% and 56.2% reductions on the most TLB-sensitive architectures.

The suite-level gains aggregate widely varying per-workload responses. THP speedups couple only loosely to baseline TLB pressure: 737.gmsh_r, the most translation-bound *SPEC CPU26* Rate workload (Figure 2), gains about as much (4-8%) as the far less translation-bound 777.zstd_r and 706.stockfish_r (5-7%). The extremes are instead set by allocator – THP interactions: under *mimalloc*+THP, 507.cactuBSSN_r (*SPEC CPU17*) and 709.cactus_r (*SPEC CPU26*) range from a 2× slowdown to a 1.26× speedup across the 4KB platforms. Page-size tuning therefore requires end-to-end validation across translation-bound, allocator-sensitive, and fragmentation-prone workloads – coverage the Rate suites provide in one run.

In contrast, systems with native 64KB pages largely avoid these translation bottlenecks, leaving little room for THP to improve performance. Across the 64KB-page AArch64 platforms, enabling THP-aware allocators changes suite-level *SPEC CPU26* scores by only -0.1% to 3.0%. The downside risk remains, however: with *mimalloc*, THP degrades both 507.cactuBSSN_r and 709.cactus_r, as it does on the 4KB-page platforms. Huge-page fragmentation and metadata

Table 6: Prefetch friendliness of *SPEC CPU26* INT Rate workloads (IPC gain from enabling all prefetchers on CPU-B).

Low ($\leq 2\%$)	Moderate (4-6%)	High ($\geq 10\%$)
707.ntest, 710.omnetpp, 735.gem5, 753.ns3	706.stockfish, 714.cpython, 723.llvm, 777.zstd	708.sqlite, 721.gcc, 727.cp- pcheck, 729.abc, 734.vpr, 750.sealcrypto

costs therefore persist, while the larger base page provides less translation overhead for THP to eliminate.

Takeaway: Native 64KB base pages effectively mitigate translation overheads. On 4KB-page systems, THP successfully reduces translation overheads, but trades that efficiency for software fragmentation risks.

4.1.3 Hardware Prefetchers. Modern CPUs rely on hardware prefetchers to hide memory latency, often at the cost of increased memory bandwidth [10, 57, 73, 92, 96]. We evaluate CPU-B (Intel Ice Lake) and CPU-E (AMD Milan) to avoid conclusions tied to a single vendor’s implementation; the remaining platforms were excluded because we lacked root access to configure their prefetchers. We measure prefetcher effects using *SPEC CPU17*, *SPEC CPU26*, and DCPerf.

Figure 7 compares IPC and per-channel memory bandwidth across four prefetcher configurations: *All-Prefetcher Off*, *L2-Prefetcher Off*, *L1-Prefetcher Off*, and *All-Prefetcher On*. On CPU-B, enabling all prefetchers improves the geometric-mean IPC by 1.06× for *SPEC CPU17*, 1.15× for *SPEC CPU26*, and 1.13× for DCPerf. The L2 prefetcher boosts IPC more than the L1 prefetcher (e.g., 1.12×

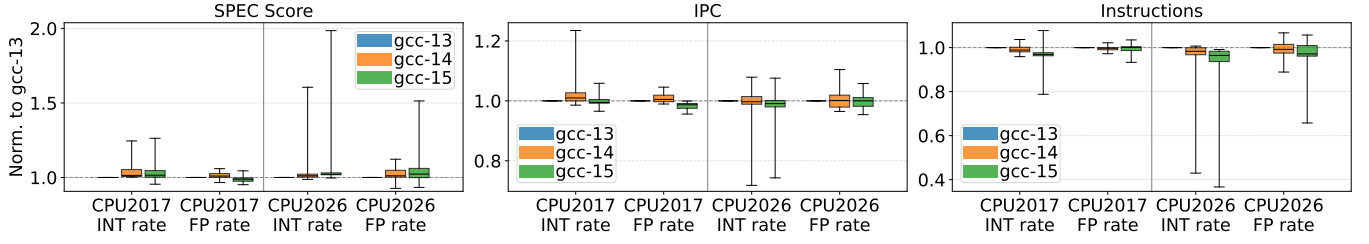


Figure 8: Performance distributions across gcc-13, gcc-14, and gcc-15 for *SPEC CPU26* and *SPEC CPU17* Rate workloads (per-workload data in Appendix Figure 17 and Figure 18). *SPEC CPU26* exhibits significantly greater performance sensitivity to compiler evolution, primarily driven by fluctuations in dynamic instruction count.

vs. $1.07\times$ for *SPEC CPU26*). In contrast, on CPU-E, the geometric-mean IPC gains are heavily muted: $1.01\times$ for *SPEC CPU17*, $1.04\times$ for *SPEC CPU26*, and $1.02\times$ for DCPerf. One reason is the higher core count on CPU-E (64 vs. 40 cores) sharing the same 3200 MT/s per-channel memory bandwidth, leaving less bandwidth headroom for speculative prefetch requests.

Despite these vendor and architectural differences, both platforms exhibit the same qualitative trend: prefetching yields substantial improvements only when sufficient memory bandwidth headroom exists. Furthermore, these gains remain highly workload-dependent; for instance, *SPEC CPU26* exhibits a significant upside in 727.cppcheck_r, where prefetching improves IPC by $1.69\times$ on CPU-B and $1.26\times$ on CPU-E.

The six *prefetch-friendly* workloads (Table 6) gain 1.18 – $1.69\times$ IPC on CPU-B. In every case, the L2 prefetcher contributes more than the L1 prefetcher; for 727.cppcheck_r, the gains are $1.57\times$ and $1.25\times$, respectively. These improvements increase DRAM traffic by up to $2.9\times$ as latency-bound misses are converted into timely prefetches. On CPU-E, the gains shrink to 1.01 – $1.26\times$ for two reasons. 727.cppcheck_r, 721.gcc_r, 708.sqlite_r, and 734.vpr_r already consume 9–16 GB/s per channel, leaving little bandwidth headroom. In contrast, 750.sealcrypto_r gains only 1.3% despite using just ~ 3.5 GB/s, because CPU-E already achieves high IPC with prefetchers disabled. Prefetching can also cause performance degradations. On CPU-B, 710.omnetpp_r loses 2% IPC while its DRAM traffic nearly triples ($0.56 \rightarrow 1.65$ GB/s), indicating that inaccurate prefetches displace useful data. Overall, CPU-B’s $1.15\times$ geometric-mean IPC gain comes at the cost of an 80% increase in DRAM traffic ($2.34 \rightarrow 4.20$ GB/s per channel).

SPEC CPU26 also excludes certain “stress-test” workloads present in *SPEC CPU17*, such as 505.mcf_r, which exert substantially higher pressure on the memory subsystem than typical datacenter applications [134]. These workloads can reach nearly 80% of the system’s peak bandwidth – approaching practical saturation [31] – while such extreme cases are often better represented by dedicated memory subsystem benchmarks such as *Mess* [16, 31].

Takeaway: *SPEC CPU26* successfully exposes prefetcher sensitivities without relying on extreme, unrealistic bandwidth stress-tests (e.g., 505.mcf_r in *SPEC CPU17*). Furthermore, cross-vendor analysis confirms that absolute prefetcher efficacy depends heavily on the platform’s core-to-bandwidth ratio rather than the workload alone.

4.1.4 Compiler. *SPEC CPU26* can be used to evaluate the effectiveness of compilers, extending *SPEC CPU17*’s long-standing role [116, 117, 128]. Comparing compiler generations across both suites quantifies whether score changes reflect improved code generation and how much optimization headroom *SPEC CPU26* restores.

We evaluate compiler sensitivity using gcc-13, gcc-14, and gcc-15. As shown in Figure 8 (distribution includes results of all nine platforms), gcc-15 improves INT Rate performance over gcc-13 for both suites, with geometric-mean speedups of $1.04\times$ on *SPEC CPU17* and $1.08\times$ on *SPEC CPU26*. For FP Rate, gcc-15 slightly regresses on *SPEC CPU17* ($0.99\times$) but improves *SPEC CPU26* ($1.05\times$). IPC and instruction counts reveal a clear contrast between the two suites. For *SPEC CPU26*, the performance gains come primarily from a 17.1% reduction in dynamic instruction count relative to gcc-13, despite a 3.5% decrease in IPC. In comparison, *SPEC CPU17* is much less sensitive, with instruction count decreasing by only 4.8% and IPC changing by just 0.1%. This may reflect diminishing returns from a decade of compiler co-evolution with *SPEC CPU17*; the fresh workloads in *SPEC CPU26* offer renewed optimization headroom. At the per-workload level, certain *SPEC CPU26* programs respond robustly to modern optimizations; for example, gcc-15 reduces the instruction count of 706.stockfish_r by up to $3\times$. This single workload forms the deep lower whisker of the *SPEC CPU26* INT rate instruction-count distribution in Figure 8 ($0.37\times$); the corresponding *SPEC CPU17* extreme, 525.x264_r, shrinks by only 21% ($0.79\times$).

Takeaway: *SPEC CPU26* restores the compiler headroom that *SPEC CPU17* exhausted, which came mainly from eliminating dynamic instructions (e.g., $3\times$ fewer instructions for 706.stockfish_r). Compiler choice is therefore a first-order experimental variable for *SPEC CPU26*: studies should report toolchain versions alongside scores and avoid frozen-binary simulation for compiler-sensitive workloads.

4.2 Evaluating Architectural Designs

We show how *SPEC CPU26* can evaluate architectural design choices, including instruction-set comparisons (x86_64 vs. AArch64, § 4.2.1) and many-core SoC design (§ 4.2.2).

4.2.1 ISA. We use *SPEC CPU26* to study ISA-level trade-offs, focusing on x86_64 (CPU-A to CPU-G) versus AArch64 (CPU-H and CPU-I) and using per-copy retired instruction count, which isolates ISA-level differences from platform-specific core and cache effects.

As shown in Figure 9, *SPEC CPU26* exhibits substantially higher cross-platform variation than *SPEC CPU17* even for INT Rate. Across

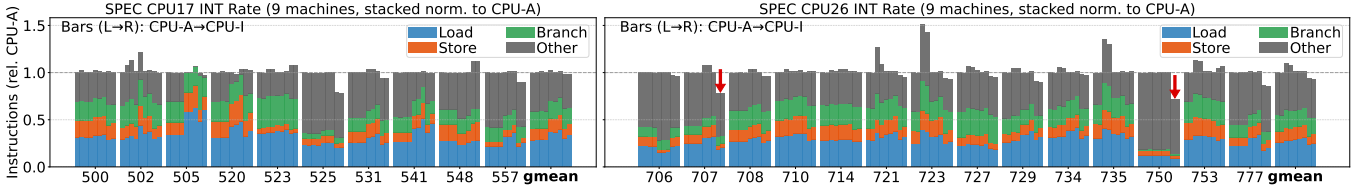


Figure 9: Per-copy retired-instruction distributions for *SPEC CPU17* and *SPEC CPU26* INT Rate across nine x86₆₄ and AArch64 systems. For each benchmark, 9 bars are pasted in each cluster for CPU-A to CPU-I. As shown by the last two bars for workloads 707 and 750, these *SPEC CPU26* workloads retire fewer instructions on AArch64 than on x86₆₄. Overall, *SPEC CPU26* exhibits greater cross-platform variation than *SPEC CPU17*.

Table 7: Workload scalability classification by scaling (40-thread score > 20× 1-thread score = High-scaling).

Suite	Low-scaling	High-scaling
<i>SPEC CPU17</i>	603.bwaves, 619.lbm, 621.wrf, 628.pop2, 649.fotonik3d, 654.roms, 657.xz	607.cactuBSSN, 627.cam4, 638.imagick, 644.nab
<i>SPEC CPU26</i>	800.pot3d, 801.xz, 809.cactus, 811.tealeaf, 817.flac, 820.cloverleaf, 822.palm, 846.minizinc, 849.fotonik3d, 865.roms, 872.marian	803.sph_exa, 807.ntest, 816.nab, 821.gcc, 823.llvm, 827.cppcheck, 838.diamond, 854.graph500, 857.namd, 867.nest, 881.neutron

all platforms, *SPEC CPU17* instruction counts span 0.78-1.22× (standard deviation 5.9%), whereas *SPEC CPU26* expands the range to 0.72-1.51× (standard deviation 10.1%). *SPEC CPU26* also contains more workloads with clear ISA-separated behavior; for example, 750.sealcrypto_r and 707.ntest_r retire approximately 20-30% fewer instructions on AArch64 than on x86₆₄, while *SPEC CPU17* has fewer cases with gaps of similar magnitude.

Takeaway: *SPEC CPU26* exhibits larger cross-ISA and cross-platform variation in instruction count than *SPEC CPU17*, making it more effective for ISA comparisons.

4.2.2 Many-Core SoC. As processor core counts have increased substantially over the past decade [4, 13, 32, 91, 94, 98, 123, 125], multi-threaded workloads have become increasingly important for evaluating many-core SoC design. Their scalability is shaped by shared on-chip resources and cross-core communication, e.g., LLC slices, coherence protocols, and memory-system traffic, all of which become even more critical with the widespread adoption of chiplet-based CPUs [89, 124, 133]. Chiplets enable modular composition and improve performance scaling, but they can introduce non-uniform latencies or reduced bandwidth across chip boundaries [34, 70, 89, 124, 133, 141, 152]. We compare CPU-B (Intel Icelake, 40-core monolithic) and CPU-C (Intel Sapphire Rapids, 4 × 12-core chiplets).

Compared with *SPEC CPU17*, which includes 11 multi-threaded Speed workloads, *SPEC CPU26* includes 22, providing broader coverage of modern many-core systems. To study many-core SoC design effects, we sweep thread counts from 1 to 40 and report Score, IPC, and L3\$ MPKI distributions in Figure 10. At 40 threads, the geometric-mean normalized Score reaches 15.31 × / 16.31 × (*SPEC CPU17*, CPU-B/CPU-C) vs. 16.53 × / 18.28 × (*SPEC CPU26*, CPU-B/CPU-C). Moreover, *SPEC CPU26* continues to scale from 32 to 40 threads (+9.5%/+10.3%), while *SPEC CPU17* shows earlier saturation (+3.1%/+5.0%). *SPEC CPU26* thus better stresses modern many-core systems.

Chiplet vs. Monolithic. CPU-C achieves 1.13× and 1.07× higher absolute geometric-mean performance than CPU-B on *SPEC CPU17*

and *SPEC CPU26* INT Rate (per-copy), respectively. However, our scaling results are normalized to a single-thread baseline, factoring out raw core improvements to isolate cross-core interactions. In *SPEC CPU17*, we observe distinct monolithic advantages for a small subset of low-scaling workloads (Table 7) that are highly sensitive to synchronization and serial cross-core communication overheads. For example, at 40 threads, 628.pop2_s achieves 11.72× on CPU-B but only 9.52× on CPU-C (a 1.23× monolithic advantage), while 621.wrf_s reaches 15.64× on CPU-B versus 12.35× on CPU-C (1.27×). These gaps persist despite CPU-C’s lower relative L3\$ MPKI increase (e.g., 628.pop2_s L3\$ MPKI at 40 threads scales by 1.97× on CPU-B vs. 1.34× on CPU-C). Non-uniform cross-chiplet coherence latencies can thus severely penalize older software patterns relying on tight synchronization, even with superior cache capacity.

In contrast, *SPEC CPU26* exhibits a significantly more balanced scaling profile. CPU-B outscales CPU-C for far fewer workloads, and the monolithic advantages are considerably narrower. The sole prominent outlier is the low-scaling 846.minizinc_s, which reaches 2.04× on CPU-B versus 1.68× on CPU-C at 40 threads (a 1.21× advantage). Beyond this single case, the performance deltas between the two topologies for the remaining low-scaling workloads fall within approximately 1-2%. Consequently, while *SPEC CPU17* highlights unscalable cases where old synchronization models favor monolithic designs, *SPEC CPU26* indicates that modern code structures are largely robust against chiplet-induced latency penalties up to 40 threads, consistent with expanded aggregate L3 capacity absorbing much of the added cross-chiplet communication cost.

Scaling beyond 40 threads: Ultra-many-core behavior. To assess whether *SPEC CPU26* eventually succumbs to the same scalability plateaus as *SPEC CPU17* on larger topologies, we extend our evaluation to 144 threads on CPU-I (Nvidia Grace Superchip, 144 cores across two sockets). Figure 11 plots the high-scaling workloads identified in Table 7. *SPEC CPU26* continues to scale significantly better than *SPEC CPU17* past 40 threads: the geometric-mean speedup of high-scaling workloads increases from 34.8× (*SPEC CPU26*) versus 30.9× (*SPEC CPU17*) at 40 threads, to 82.1× versus 64.1× at 144 threads, a relative scalability gain of +136% versus +107%. Consequently, *SPEC CPU26*’s higher high-scaling workload fraction provides a more rigorous and rugged vehicle for evaluating ultra-many-core fabrics.

Takeaway: (1) *SPEC CPU26* is better suited to many-core SoC evaluation: it doubles the number of multi-threaded Speed workloads and maintains strong scalability into ultra-many-core (144-thread) regimes, where *SPEC CPU17* begins to plateau. (2) *SPEC CPU26* also

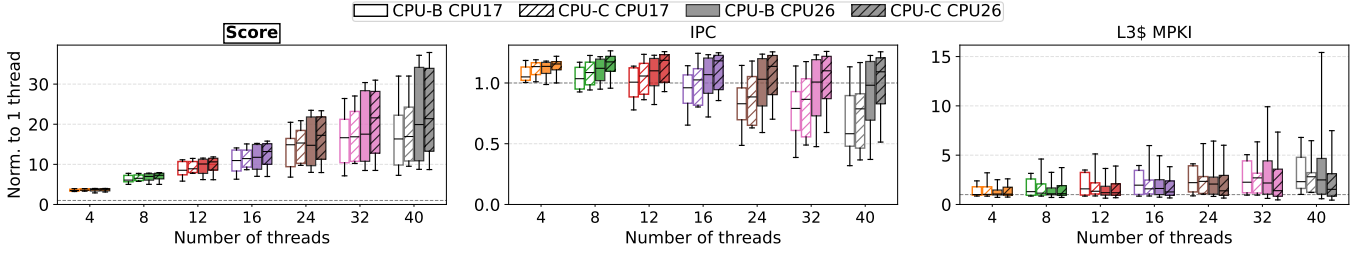


Figure 10: Scaling distributions from 1 to 40 threads for *SPEC CPU26* and *SPEC CPU17* Speed (per-workload data in Appendix Figure 19 and Figure 20) on CPU-B (40-core monolithic) and CPU-C (4x12-core chiplet). *SPEC CPU26* scales better overall, while CPU-C outperforms CPU-B due to its larger aggregate LLC. *SPEC CPU17* more often exposes communication-bound workloads that scale poorly and favor monolithic designs.

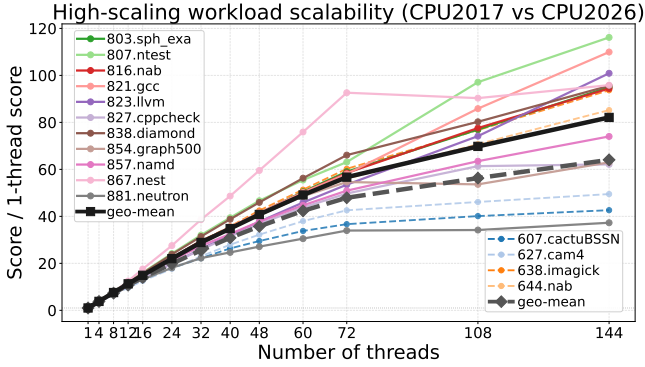


Figure 11: Scalability up to 144 threads (CPU-I). High-scalability workloads in *SPEC CPU26* continue to scale significantly better than *SPEC CPU17* past 40 threads into the ultra-many-core regime.

avoids extreme scalability anomalies. Its only apparent monolithic-topology advantage, 846.minizinc_s, saturates by four threads on both systems, indicating limited parallelism rather than coherence sensitivity. Thus, architects studying cross-chiplet coherence should retain *SPEC CPU17*’s 628.pop2_s and 621.wrf_s as targeted probes, while using *SPEC CPU26* to evaluate the strengths of chiplet designs, including larger aggregate LLC capacity (Figure 10) and sustained high-concurrency scaling.

4.3 Creating Workload Proxies Using RRR Mode

Standard *SPEC CPU26* Rate execution is homogeneous: all copies run the same benchmark before the suite advances. *SPEC CPU26*’s new Rolling Round-Robin (RRR) mode instead staggers N benchmarks across M cores, each core executing the same sequence at a different phase [56, 78, 101, 157], producing a time-rolled heterogeneous mix.

We use RRR as a composition primitive for creating proxy workloads. Full-suite RRR closely reproduces homogeneous Rate execution [80], because suite-wide averaging smooths workload-specific extremes. Targeted RRR mixes instead combine complementary workloads into controlled stress points that bracket a target pressure point. We evaluate this workflow in three steps:

Algorithm 1: Targeted RRR proxy construction.

Input : target profile t ; declared axes K ; pool $\mathcal{P}$ of solo *SPEC CPU26* profiles v^X with instruction counts I_X ; slot budget S
Output: pair (A, B) , slot weights $w_A:w_B$, unreachable axes U
 /* mixing A and B reaches only the segment $v^{AB}(\alpha) = \alpha v^A + (1-\alpha)v^B$ */
 1 $C \leftarrow$ candidate pairs (A, B) from $\mathcal{P}$ with $\min(v_k^A, v_k^B) \leq t_k \leq \max(v_k^A, v_k^B) \forall k \in K$
 2 **foreach** $(A, B) \in C$ **do**
 3 $\alpha_{AB} \leftarrow \arg \min_{0 \leq \alpha \leq 1} \sum_{k \in K} (v_k^{AB}(\alpha) / t_k - 1)^2$
 4 $(A, B) \leftarrow$ the best-fitting pair; $\alpha^* \leftarrow \alpha_{AB}$
 5 $w_A:w_B \leftarrow$ integer slots, $w_A + w_B \leq S$, with instruction share nearest α^*
 6 $U \leftarrow$ axes outside K on which (A, B) does not bracket t
 7 Run RRR at $w_A:w_B$; report the measured residual and U

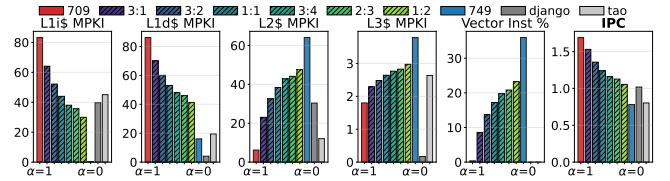


Figure 12: The RRR mix ratio is a tuning knob (CPU-C): hatched bars sweep the 709.cactus_r:749.fotonik3d_r slot ratio from 3:1 to 1:2, lowering 709.cactus_r’s instruction share α from 0.77 to 0.36. The sweep brackets *django*’s L1I\$ MPKI between the 1:1 and 3:4 mixes ($\alpha = 0.53$ and 0.45 ; 44.0 and 38.0 MPKI vs. 39.7).

- Derive a workload pair and mix ratio that bracket a desired target pressure point (Alg. 1).
- Test whether RRR mixes follow predictions from weighted solo profiles.
- Quantify the contention omitted by time-rolled execution through true co-location experiments.

Bracketing a target profile. Alg. 1 derives both the workload pair and its mix ratio from a first-order composition model. An RRR mix retires the full instruction stream of each constituent to within 0.02%, so every per-instruction metric is linear in the instruction share $\alpha = I_A / (I_A + I_B)$, and IPC composes as the weighted harmonic mean $\text{IPC}_{\text{mix}} = (I_A + I_B) / (C_A + C_B) = (\alpha / \text{IPC}_A + (1-\alpha) / \text{IPC}_B)^{-1}$. Line 1 enumerates the pairs in the pool and keeps those whose solo values straddle the target on every declared axis $k \in K$. Because the achievable set is the segment, a pair that fails to straddle t_k

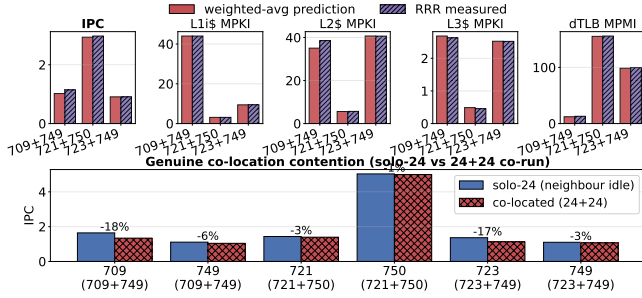


Figure 13: RRR mixes and true co-location on CPU-C. Top: the measured RRR mix against the instruction-weighted prediction built from the two solo profiles, for pairs 709+749, 721+750, and 723+749. Bottom: each workload’s IPC with its partner running on the other 24 cores versus with that half idle (loss annotated per bar).

cannot reach it at any ratio, so bracketing is a necessary condition and is cheap enough to test exhaustively at suite scale. Lines 2-3 fit each surviving pair, choosing the instruction share that minimizes the squared *relative* deviation from the target, so that axes of very different magnitude count equally. Entering IPC as its reciprocal keeps every axis linear in α , leaving a one-dimensional quadratic that is solved in closed form and clipped to $[0, 1]$. Line 4 takes the pair with the smallest residual, and line 5 converts its α^* into whole slots, since RRR schedules entire benchmark slots rather than fractional shares. The search is over integer weights within the slot budget S whose instruction share $w_A I_A / (w_A I_A + w_B I_B)$ is nearest α^* , so the rounding tracks instructions rather than slot counts. Line 6 records the axes outside K that the chosen pair cannot span, the proxy’s fidelity bound, and line 7 measures the composed mix, because the weighted model is a prediction rather than a guarantee.

We apply this procedure to 26 *SPEC CPU26* Rate benchmarks profiled at 48 copies, targeting DCPerf *django*’s L1I\$ MPKI and IPC (Figure 2). The bracketing requirement sharply narrows the candidates. 709.cactus_r is the only benchmark whose L1I\$ MPKI (82.3) exceeds *django*’s 39.7, while only 721.gcc_r and 749.fotonik3d_r run more slowly than *django* (IPC 1.02). The fit selects 749.fotonik3d_r (IPC 0.79), which has high L2 pressure and vector intensity (64.1 L2\$ MPKI and 36% vector instructions) but negligible L1I\$ pressure (0.3 MPKI). The resulting 709.cactus_r + 749.fotonik3d_r proxy, run at stock RRR’s default 1:1 slot ratio, lies between its constituents on the measured metrics (Figure 12, CPU-C): L1I\$ MPKI 43.9, L2\$/L3\$ MPKI 38.5/2.64, and IPC 1.16. It lands between *django*’s and *tao*’s L1I\$ MPKI (39.7 and 45.1) at an IPC comparable to *django*’s 1.02. It lands between *django*’s and *tao*’s L1I\$ MPKI (39.7 and 45.1), while its IPC overshoots *django*’s 1.02 by 14%.

he mix ratio is likewise derived, not assumed. The default 1:1 mix overshoots *django*’s L1I\$ MPKI (44.0 vs. 39.7) and 3:4 undershoots it (38.0), so the ratio matching *django* on both declared axes lies between them – exactly where the model places its optimum, at $\alpha = 0.476$ against the 0.53 and 0.45 of those two mixes (Figure 12). Because stock RRR assigns one slot per benchmark, we added a per-benchmark slot-weight option for profiling only; it is not used for reportable *SPEC CPU26* scores. The procedure also bounds its own fidelity by reporting unreachable axes: both candidate pairs bottom

out at L1D\$ MPKI 10.7 and 16.1 (721.gcc_r and 749.fotonik3d_r), far above *django*’s 4.1.

Generalization across pairs. We repeat the analysis with two additional pairs, 723.llvm_r + 749.fotonik3d_r and 721.gcc_r + 750.sealcrypto_r. The measured RRR profiles generally follow the instruction-weighted solo predictions (Figure 13, top). Weighting by instructions matters for asymmetric pairs: 721.gcc_r + 750.sealcrypto_r measures 156 dTLB MPMI and is predicted at 155, versus an unweighted midpoint of 279. Only four of the fifteen plotted quantities depart by more than 5%: IPC, L2\$ MPKI, and dTLB MPMI for 709.cactus_r + 749.fotonik3d_r (+12%, +10%, and +7%), and L3\$ MPKI for 721.gcc_r + 750.sealcrypto_r (−7%). Weighted solo profiles thus provide useful first-order estimates, but the constructed mix must still be measured.

Time-rolled RRR vs. true co-location. What makes RRR predictable also limits it: because RRR mixes workloads over time rather than running them together, a mix tracks the instruction-weighted average of its constituents and does not capture the contention that simultaneous tenants impose on one another [38, 59]. The 709.cactus_r + 749.fotonik3d_r mix, in fact, exceeds its instruction-weighted prediction by 12% in IPC (Figure 13, top): only part of the machine runs the memory-streaming 749.fotonik3d_r at any instant, so mixing eases contention rather than adding it. To measure the omitted contention, we partition CPU-C’s 48 cores into two disjoint 24-core halves and run one workload of a pair on each half at the same time, 24 copies per half. The baseline reruns each workload on its own half with the other half idle (Figure 13, bottom), so copy count and core count are unchanged and the difference isolates shared-cache and memory interference.

The interference is strongly asymmetric. Streaming 749.fotonik3d_r costs its partners 18% (709.cactus_r) and 17% (723.llvm_r) of their IPC while losing only 3-6% itself; compute-bound 750.sealcrypto_r is almost immune (−1%) and costs 721.gcc_r only 3%. A proxy’s disruptiveness is therefore set by its memory demand rather than the share of the machine it occupies, and it must be reported per tenant rather than as a single number for the pair.

Takeaway: RRR can be used to construct desired microarchitectural proxies by combining distinct *SPEC CPU26* workloads. A target profile and declared axes determine the initial workload pair and mix ratio, and measured mixes can be tuned to achieve desired goals. However, because RRR time-rolls its constituents instead of running them simultaneously, it generates proxies that may increase or decrease contention; true co-location effects may require separate measurement.

5 Representative Workloads in *SPEC CPU26*

This section applies the hierarchical clustering method of § 3.2 to identify representative subsets that preserve the full *SPEC CPU26* suite’s behavioral diversity. We cut each dendrogram at a fixed linkage distance to form behaviorally similar groups; Figure 14 illustrates this process for the INT/FP Rate/Speed suites in *SPEC CPU26*. Within each group, we select the medoid, i.e., the workload with minimum average distance to other group members in PCA space. To quantify subset fidelity, we define subset *accuracy* as $1 - \text{err}$, where $\text{err} = |\text{GM}(\text{subset}) - \text{GM}(\text{suite})| / \text{GM}(\text{suite})$, and $\text{GM}(\cdot)$

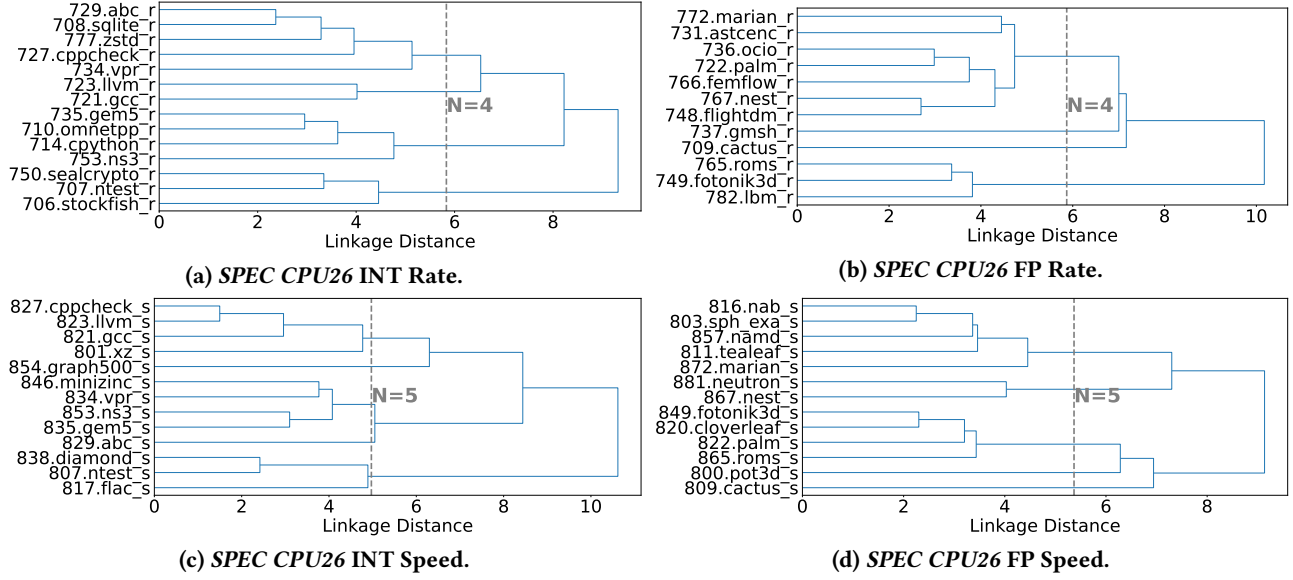


Figure 14: Dendrogram showing similarity between *SPEC CPU26* workloads; shorter linkage distance indicates higher similarity.

Table 8: Representative subset per group and accuracy.

Group	Subset workloads	Accuracy
int_rate	750.sealcrypto_r, 735.gem5_r, 721.gcc_r, 708.sqlite_r	99.90%
fp_rate	749.fotonik3d_r, 709.cactus_r, 737.gmsh_r, 767.nest_r	99.07%
int_speed	807.ntest_s, 854.graph500_s, 823.llvm_s, 829.abc_s, 835.gem5_s	96.61%
fp_speed	867.nest_s, 816.nab_s, 809.cactus_s, 800.pot3d_s, 849.fotonik3d_s	96.40%

is the geometric-mean score of *SPEC CPU26*. Table 8 summarizes the subsets for all four *SPEC CPU26* groups.

Takeaway: Compact subsets of 4-5 workloads per group achieve 96.4-99.9% accuracy, substantially reducing evaluation cost while preserving representative behavior.

6 Conclusion

This paper presents a comprehensive characterization of the just-released *SPEC CPU26* benchmark suite. Across nine platforms, *SPEC CPU26* broadens coverage over *SPEC CPU17* through larger instruction volumes and memory footprints and greater pressure on CPU bottlenecks, especially the instruction cache. Comparisons with *SPEC CPU17*, DCPerf, and MLPerf show that *SPEC CPU26* remains a general-purpose suite with proximity to foundational datacenter workloads while staying distinct from vector-centric MLPerf inference; an agentic AI pipeline falls within that coverage, while adding a dense GEMM kernel reaches only MLPerf’s dense-kernel corner rather than closing the gap to MLPerf. Our case studies demonstrate value for many explorations, including page size tradeoffs, prefetching, compiler sensitivity, and scalability, making *SPEC CPU26* a forward-looking complement to domain-specific suites.

Acknowledgment

The authors would like to thank the *SPEC CPU* Subcommittee for the opportunity to collaborate on the clustering analysis for the *SPEC CPU26* program selection. We are grateful to the member companies – specifically AMD, Ampere, IBM, and Intel – for providing the necessary data. Special thanks go to John Henning, Mahesh Madhav, and Frédérique Silber-Chaussumier for their invaluable technical support and for facilitating the analysis. Additionally, we thank the Texas Advanced Computing Center (TACC) at the University of Texas at Austin for providing the computing resources that supported this work.

References

- [1] Siddharth Agarwal, Tianchen Wang, Jinghan Huang, Saksham Agarwal, and Nam Sung Kim. 2026. TiNA: Tiered Network Buffer Architecture for Fast Networking in Chiplet-based CPUs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 298–313.
- [2] Manoj Alwani, Han Chen, Michael Ferdman, and Peter Milder. 2016. Fused-layer CNN accelerators. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
- [3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [4] Ampere. 2026. Ampere Processor Platforms. <https://amperecomputing.com/products/processors>.
- [5] Ulf Andersson, Min Qiu, and Ziyang Zhang. 2006. Parallel power computation for photonic crystal devices. *Methods and applications of analysis* 13, 2 (2006), 149–156.
- [6] Georgia Antoniou, Davide Bartolini, Haris Volos, Marios Kleanthous, Zhe Wang, Kleovoulos Kalaitzidis, Tom Rollet, Ziwei Li, Onur Mutlu, Yiannakis Sazeides, and Jawad Haj Yahya. 2024. Agile C-states: a core C-state architecture for latency critical applications optimizing both transition and cold-start latency. *ACM Transactions on Architecture and Code Optimization* 21, 4 (2024), 1–26.
- [7] ARM. 2026. The Arm ASTC Encoder, a compressor for the Adaptive Scalable Texture Compression data format. <https://github.com/ARM-software/astc-encoder>.
- [8] ARM. 2026. The world’s most efficient agentic CPU. <https://www.arm.com/products/cloud-datacenter/arm-agi-cpu>.

- [9] Rachata Ausavarungrun, Joshua Landgraf, Vance Miller, Saugata Ghose, Jayneel Gandhi, Christopher J Rossbach, and Onur Mutlu. 2018. Mosaic: Enabling application-transparent support for multiple page sizes in throughput processors. *ACM SIGOPS Operating Systems Review* 52, 1 (2018), 27–44.
- [10] Mohammad Bakhshalipour, Seyedal Tabaeiaghdaei, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Evaluation of hardware data prefetchers on server processors. *ACM Computing Surveys (CSUR)* 52, 3 (2019), 1–29.
- [11] Jon Berndt. 2004. JSBSim: An open source flight dynamics model in C++. In *AIAA modeling and simulation technologies conference and exhibit*. 4923.
- [12] Vaughn Betz and Jonathan Rose. 1997. VPR: A new packing, placement and routing tool for FPGA research. In *International Workshop on Field Programmable Logic and Applications*. Springer, 213–222.
- [13] Ravi Bhargava and Kai Troester. 2024. AMD next-generation “Zen 4” core and 4th gen AMD EPYC server CPUs. *IEEE Micro* 44, 3 (2024), 8–17.
- [14] Tawhid Bhuiyan, Sumya Hoque, Angélica Aparecida Moreira, and Tanvir Ahmed Khan. 2026. Wax: Optimizing Data Center Applications With Stale Profile. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 2232–2248.
- [15] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. 2011. The gem5 simulator. *ACM SIGARCH computer architecture news* 39, 2 (2011), 1–7.
- [16] F. Nisa Bostanci, Haocong Luo, Ataberk Olgun, Maria Makeenkova, Gerardo F. Oliveira, A. Giray Yaglikci, and Onur Mutlu. 2026. Cleaning up the Mess: Re-Evaluating the Real-System Modeling Accuracy of Ramulator 2.0. arXiv:2510.15744 [cs.AR] <https://arxiv.org/abs/2510.15744>
- [17] Robert Brayton and Alan Mishchenko. 2010. ABC: An academic industrial-strength verification tool. In *International Conference on Computer Aided Verification*. Springer, 24–40.
- [18] James Bucek, Klaus-Dieter Lange, and J  akim v. Kistowski. 2018. SPEC CPU2017: Next-generation compute benchmark. In *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*. 41–42.
- [19] Benjamin Buchfink, Klaus Reuter, and Hajk-Georg Drost. 2021. Sensitive protein alignments at tree-of-life scale using DIAMOND. *Nature methods* 18, 4 (2021), 366–368.
- [20] Calin Cascaval, Evelyn Duesterwald, Peter F Sweeney, and Robert W Wisniewski. 2005. Multiple page size modeling and optimization. In *14th International Conference on Parallel Architectures and Compilation Techniques (PACT’05)*. IEEE, 339–349.
- [21] Hao Chen, Kim Laine, and Rachel Player. 2017. Simple encrypted arithmetic library-SEAL v2. 1. In *International conference on financial cryptography and data security*. Springer, 3–18.
- [22] Hongzheng Chen, Jiahao Zhang, Yixiao Du, Shaojie Xiang, Zichao Yue, Niansong Zhang, Yaohui Cai, and Zhiru Zhang. 2024. Understanding the potential of fpga-based spatial acceleration for large language model inference. *ACM Transactions on Reconfigurable Technology and Systems* 18, 1 (2024), 1–29.
- [23] Shuang Chen, Christina Delimitrou, and Jos   F Mart  nez. 2019. Parties: Qos-aware resource partitioning for multiple interactive services. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 107–120.
- [24] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits* 52, 1 (2016), 127–138.
- [25] Weiwei Chu, Xinfeng Xie, Jiecao Yu, Jie Wang, Amar Phanishayee, Chunqiang Tang, Yuchen Hao, Jianyu Huang, Mustafa Ozdal, Jun Wang, Vedanuj Goswami, Naman Goyal, Abhishek Kadian, Andrew Gu, Chris Cai, Feng Tian, Xiaodong Wang, Min Si, Pavan Balaji, Ching-Hsiang Chu, and Jongsoo Park. 2025. Scaling Llama 3 Training with Efficient Parallelism Strategies. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1703–1716.
- [26] Joel Coburn, Chunqiang Tang, Sameer Abu Asal, Neeraj Agrawal, Raviteja Chinta, Harish Dixit, Brian Dodds, Saritha Dwarakapuram, Amin Firoozshahian, Cao Gao, Kaustubh Gondkar, Tyler Graf, Junhan Hu, Jian Huang, Sterling Hughes, Adam Hutchin, Bhasker Jakka, Guoqiang Jerry Chen, Indu Kalyanaraman, Ashwin Kamath, Pankaj Kansal, Erum Kazi, Roman Levenstein, Mahesh Maddury, Alex Mastro, Siji Medaiyese, Pritesh Modi, Jack Montgomery, Satish Nadathur, Amit Nagpal, Ashwin Narasimha, Maxim Naumov, Eleanor Ozer, Jongsoo Park, Poorvaja Ramani, Harikrishna Reddy, David Reiss, Deboleena Roy, Sathish Sekar, Arushi Sharma, Pavan Shetty, Aravind Sukumaran-Rajam, Eran Tal, Mike Tsai, Shreya Varshini, Richard Wareing, Olivia Wu, Xiaolong Xie, Jinghan Yang, Hangchen Yu, Tanmay Zargar, Zitong Zeng, Feixiong Zhang, Ajit Matthews, Xun Jiao, Jiyuan Zhang, Emmanuel Menage, Truls Edvard Stokke, and Mohammed Sourouri. 2025. Meta’s Second Generation AI Chip: Model-Chip Co-Design and Productionization Experiences. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1689–1702.
- [27] Guilherme Cox and Abhishek Bhattacharjee. 2017. Efficient address translation for architectures with multiple page sizes. *ACM SIGPLAN Notices* 52, 4 (2017), 435–448.
- [28] Peter J Denning. 1968. The working set model for program behavior. *Commun. ACM* 11, 5 (1968), 323–333.
- [29] Kaivalya M Dixit. 1991. The SPEC benchmarks. *Parallel Comput.* 17, 10-11 (1991), 1195–1209.
- [30] Cooper Downs, Jon A. Linker, Ronald M. Caplan, Emily I. Mason, Pete Riley, Ryder Davidson, Andres Reyes, Erika Palmerio, Roberto Lionello, James Turtle, Michal Ben-Nun, Miko M. Stulajter, Viacheslav S. Titov, Tibor T  r  k, Lisa A. Upton, Raphael Attie, Bibhuti K. Jha, Charles N. Arge, Carl J. Henney, Gherardo Valori, Hanna Strecker, Daniele Calchetti, Dietmar Germerott, Johann Hirzberger, David Orozco Su  rez, Julian Blanco Rodr  guez, Sami K. Solanki, Xin Cheng, and Sizhe Wu. 2025. A near-real-time data-assimilative model of the solar corona. *Science* 388, 6753 (2025), 1306–1310.
- [31] Pouya Esmaili-Dokht, Francesco Sgherzi, Val  ria Soldera Girelli, Isaac Boixaderas, Mariana Carmin, Alireza Monemi, Adri   Armeljach, Estandisao Mercadal, Germ  n Llort, Petar Radojkovi  , Miquel Moreto, Judit Gim  nez, Xavier Martorell, Eduard Ayguad  , Jesus Labarta, Emanuele Confalonieri, Rishabh Dubey, and Jason Adlard. 2024. A mess of memory system benchmarking, simulation and application profiling. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 136–152.
- [32] Mark Evers, Leslie Barnes, and Mike Clark. 2022. The AMD next-generation “Zen 3” core. *IEEE Micro* 42, 3 (2022), 7–12.
- [33] Facebook. 2026. Zstandard - Fast real-time compression algorithm. <https://github.com/facebook/zstd>.
- [34] Yinxiao Feng and Kaisheng Ma. 2022. Chiplet actuary: A quantitative cost model and multi-chiplet architecture exploration. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 121–126.
- [35] Amin Firoozshahian, Joel Coburn, Roman Levenstein, Rakesh Nattoji, Ashwin Kamath, Olivia Wu, Gurdeepak Grewal, Harish Aepala, Bhasker Jakka, Bob Dreyer, Adam Hutchin, Utku Diril, Krishnakumar Nair, Ehsan K. Aredastani, Martin Schatz, Yuchen Hao, Rakesh Komuravelli, Kunming Ho, Sameer Abu Asal, Joe Shajrawi, Kevin Quinn, Nagesh Sreedhara, Pankaj Kansal, Willie Wei, Dheepak Jayaraman, Linda Cheng, Pritam Chopda, Eric Wang, Ajay Bikumandla, Arun Karthik Sengottuvel, Krishna Thottempudi, Ashwin Narasimha, Brian Dodds, Cao Gao, Jiyuan Zhang, Mohammed Al-Sanabani, Ana Zehtabioskuie, Jordan Fix, Hangchen Yu, Richard Li, Kaustubh Gondkar, Jack Montgomery, Mike Tsai, Saritha Dwarakapuram, Sanjay Desai, Nili Avidan, Poorvaja Ramani, Karthik Narayanan, Ajit Mathews, Sethu Gopal, Maxim Naumov, Vijay Rao, Krishna Noru, Harikrishna Reddy, Prahlad Venkatapuram, and Alexis Bj  rlin. 2023. Mta: First generation silicon targeting meta’s recommendation systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [36] Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Todd Massengill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, Logan Adams, Mahdi Ghandi, Stephen Heil, Prerak Patel, Adam Sapke, Gabriel Weisz, Lisa Woods, Sitarum Lanka, Steven K. Reinhardt, Adrian M. Caulfield, Eric S. Chung, and Doug Burger. 2018. A configurable cloud-scale DNN processor for real-time AI. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1–14.
- [37] Kevin P Gaffney, Martin Prammer, Larry Brasfield, D Richard Hipp, Dan Kennedy, and Jignesh M Patel. 2022. SQLite: past, present, and future. *Proceedings of the VLDB Endowment* 15, 12 (2022).
- [38] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Cole, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvisky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 3–18.
- [39] Christophe Geuzaine and Jean-Fran  ois Remacle. 2009. Gmsh: A 3-D finite element mesh generator with built-in pre-and post-processing facilities. *International journal for numerical methods in engineering* 79, 11 (2009), 1309–1331.
- [40] Marc-Oliver Gewaltig and Markus Diesmann. 2007. Nest (neural simulation tool). *Scholarpedia* 2, 4 (2007), 1430.
- [41] Abraham Gonzalez, Aasheesh Kolli, Samira Khan, Sihang Liu, Vidushi Dadu, Sagar Karandikar, Jichuan Chang, Krste Asanovic, and Parthasarathy Ranganathan. 2023. Profiling hyperscale big data processing. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–16.
- [42] Tom Goodale, Gabrielle Allen, Gerd Lanfermann, Joan Mass  , Thomas Radke, Edward Seidel, and John Shalf. 2002. The cactus framework and toolkit: design and applications: invited talk. In *International conference on high performance computing for computational science*. Springer, 197–227.
- [43] Kazushige Goto and Robert A van de Geijn. 2008. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software (TOMS)* 34, 3 (2008), 1–25.
- [44] Bj  rn Gottschall, Silvio Campelo de Santana, and Magnus Jahre. 2023. Balancing accuracy and evaluation overhead in simulation point selection. In *2023 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 43–53.

- [45] Darryl Gove. 2007. CPU2006 working set size. *ACM SIGARCH Computer Architecture News* 35, 1 (2007), 90–96.
- [46] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esibov, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Janos Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Wang, Jiacon Li, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yearry, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Celebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shalohang Nie, Sharan Narang, Sharath Raparthi, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collet, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkze, Vincent Gnanapathy, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuwei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testugine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damla, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhee, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabza, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangarabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Zhang, Russ Howes, Rutu Rinott, Sachin Mehta, Sachin Sibi, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Mathers, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihalescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [47] Danilo Guerrero, Rubén M Cabezon, Jean-Guillaume Piccinali, Aurélien Cavelan, Florina M Ciorba, David Imbert, Lucio Mayer, and Darren Reed. 2018. Towards a mini-app for smoothed particle hydrodynamics at exascale. In *2018 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 607–614.
- [48] Faruk Guvenilir and Yale N Patt. 2020. Tailored page sizes. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 900–912.
- [49] Ranjan Hebbar SR and Aleksandar Milenković. 2019. SPEC CPU2017: Performance, event, and energy characterization on the core i7-8700K. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*. 111–118.
- [50] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. *Advances in Neural Information Processing Systems (NeurIPS)* (2021).
- [51] John L Henning. 2002. SPEC CPU2000: Measuring CPU performance in the new millennium. *Computer* 33, 7 (2002), 28–35.
- [52] John L Henning. 2006. SPEC CPU2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News* 34, 4 (2006), 1–17.
- [53] Andrew Hamilton Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. 2021. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*. 257–273.
- [54] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. Floatpim: In-memory acceleration of deep neural network training with high precision. In *Proceedings of the 46th International Symposium on Computer Architecture*. 802–815.
- [55] Koki Ishida, Ilkwon Byun, Ikki Nagaoka, Kosuke Fukumitsu, Masamitsu Tanaka, Satoshi Kawakami, Teruo Tanimoto, Takatsugu Ono, Jangwoo Kim, and Koji Inoue. 2020. SuperNPU: An extremely fast neural processing unit using superconducting logic devices. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 58–72.
- [56] Adam N Jacobvitz, Andrew D Hilton, and Daniel J Sorin. 2015. Multi-program benchmark definition. In *2015 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 72–82.
- [57] Akanksha Jain, Hannah Lin, Carlos Villavieja, Baris Kasikci, Chris Kennelly, Milad Hashemi, and Parthasarathy Ranganathan. 2024. Limoncello: Prefetchers for scale. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 577–590.
- [58] Rishabh Jain, Scott Cheng, Vishwas Kalagi, Vrushabh Sanghavi, Samvit Kaul, Meena Arunachalam, Kiwan Maeng, Adwait Jog, Anand Sivasubramaniam,

- Mahmut Taylan Kandemir, and Chita R. Das. 2023. Optimizing cpu performance for recommendation systems at-scale. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [59] Congfeng Jiang, Yitao Qiu, Weisong Shi, Zhefeng Ge, Jiwei Wang, Shenglei Chen, Christophe Cérin, Zujie Ren, Guoyao Xu, and Jiangbin Lin. 2020. Characterizing co-located workloads in alibaba cloud datacenters. *IEEE Transactions on Cloud Computing* 10, 4 (2020), 2381–2397.
- [60] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2025. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *Proceedings of Machine Learning and Systems* 7 (2025).
- [61] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.
- [62] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Andrew Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snellman, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [63] Jowi Morales. 2026. Are we staring down the barrel of an AI-driven CPU shortage? <https://www.tomshardware.com/pc-components/cpus/cpus-are-cool-again-intel-and-amd-reporting-spikes-in-cpu-demand-due-to-agentic-ai-shortages-lisa-su-says-business-exceeded-expectations-while-intel-is-looking-at-long-term-agreements-with-potential-customers>.
- [64] Marcin Junczys-Dowmunt, Roman Grundkiewicz, Tomasz Dwojak, Hieu Hoang, Kenneth Heafield, Tom Neckermann, Frank Seide, Ulrich Germann, Alham Fikri Aji, Nikolay Bogoychev, André F. T. Martins, and Alexandra Birch. 2018. Marian: Fast neural machine translation in C++. In *Proceedings of ACL 2018, system demonstrations*. 116–121.
- [65] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tapp Moseley, Gu-Yeon Wei, and David Brooks. 2015. Profiling a warehouse-scale computer. In *Proceedings of the 42nd annual international symposium on computer architecture*. 158–169.
- [66] Sagar Karandikar, Howard Mao, Donggyu Kim, David Biancolin, Alon Amid, Dayeol Lee, Nathan Pemberton, Emmanuel Amaro, Colin Schmidt, Aditya Chopra, Qijiang Huang, Kyle Kovacs, Borivoje Nikolic, Randy Katz, Jonathan Bachrach, and Krste Asanović. 2018. FireSim: FPGA-accelerated cycle-accurate scale-out system simulation in the public cloud. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 29–42.
- [67] Tushar Khot, Peter Clark, Michal Guerquin, Peter Jansen, and Ashish Sabharwal. 2020. Qasc: A dataset for question answering via sentence composition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 8082–8090.
- [68] Joshua Kim, Chaojie Zhang, Inigo Goiri, Christopher J Rossbach, and Jovan Stojkovic. 2026. PhaseWeave: Phase-Aware Execution on Heterogeneous Chiplet Architectures for Datacenters. (2026).
- [69] Martin Kronbichler, Dmytro Sashko, and Peter Munch. 2023. Enhancing data locality of the conjugate gradient method for high-order matrix-free finite-element implementations. *The International Journal of High Performance Computing Applications* 37, 2 (2023), 61–81.
- [70] Jaewon Kwon, Yongju Lee, Hongju Kal, Minjae Kim, Youngsok Kim, and Wonwoo Ro. 2023. McCore: A Holistic Management of High-Performance Heterogeneous Multicores. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1044–1058.
- [71] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [72] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 75–86.
- [73] Jaekyu Lee, Hyesoon Kim, and Richard Vuduc. 2012. When prefetching works, when it doesn't, and why. *ACM Transactions on Architecture and Code Optimization (TACO)* 9, 1 (2012), 1–29.
- [74] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. 2023. Memtis: Efficient memory tiering with dynamic page classification and page size determination. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 17–34.
- [75] Daan Leijen, Benjamin Zorn, and Leonardo De Moura. 2019. Mimalloc: Free list sharding in action. In *Asian Symposium on Programming Languages and Systems*. Springer, 244–265.
- [76] Ankur Limaye and Tosiron Adegbiya. 2018. A workload characterization of the spec cpu2017 benchmark suite. In *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 149–158.
- [77] Linux Kernel Community. 2026. Linux perf tool. https://perf.wiki.kernel.org/index.php/Main_Page.
- [78] Qiuyun Llull, Songchun Fan, Seyed Majid Zahedi, and Benjamin C Lee. 2017. Cooper: Task colocation with cooperative games. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 421–432.
- [79] Thomas J Macke and David A Case. 1998. Modeling unusual nucleic acid structures. ACS Publications.
- [80] Mahesh Madhav, Allen Lee, Andres Mejia, Branden Moore, Charan Sopadandi, Chris Cambly, Christoph Müllner, Daniel Bowers, David Reiner, Denis Bakhvalov, Di Zhao, Duane Voth, Feng Xue, Frédérique Silber-Chaussumier, James Bucek, James Southern, Jiangning Liu, Jim Himer, John Henning, Kevin Smith, Kristen Yang, Kunal Kashyap, Mason Guy, Mat Colgrove, Michael Berg, Prasad Battini, Prasad Joshi, Rohit Prasad, Shayanika Bhattacharya, Sriyash Caculo, Stefan Reimbold, Sundar Iyengar, Van Smith, and Zarko Todorovski. 2026. SPEC CPU: The Next Generation. In *2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 671–687.
- [81] Artemiy Margaritov, Siddharth Gupta, Reka Gonzalez-Alberquilla, and Boris Grot. 2019. Stretch: Balancing qos and throughput for collocated server workloads on smt cores. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 15–27.
- [82] Daniel Marjamäki. 2013. Cppcheck: a tool for static c/c++ code analysis. URL: <https://cppcheck.sourceforge.io> (2013).
- [83] B. Maronga, S. Banzhaf, C. Burmeister, T. Esch, R. Forkel, D. Fröhlich, V. Fuka, K. F. Gehrke, J. Geletić, S. Giersch, T. Gronemeier, G. Groß, W. Heldens, A. Hellsten, F. Hoffmann, A. Inagaki, E. Kadasch, F. Kanani-Sühring, K. Ketelsen, B. A. Khan, C. Knigge, H. Knoop, P. Krč, M. Kurppa, H. Maamari, A. Matzarakis, M. Maeder, M. Pallasch, D. Pavlik, J. Pfaffert, J. Resler, S. Rissmann, E. Russo, M. Salim, M. Schrempf, J. Schwenkel, G. Seckmeyer, S. Schubert, M. Sühring, R. von Tils, L. Vollmer, S. Ward, B. Witha, H. Wurps, J. Zeidler, and S. Raasch. 2020. Overview of the PALM model system 6.0. *Geoscientific Model Development* 13, 3 (2020), 1335–1372.
- [84] Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St. John, Tsuguchika Tabaru, Carole-Jean Wu, Lingjie Xu, Masafumi Yamazaki, Cliff Young, and Matei Zaharia. 2020. Mlperf training benchmark. *Proceedings of Machine Learning and Systems* 2 (2020), 336–349.
- [85] Simon McIntosh-Smith, Matthew Martineau, Tom Deakin, Grzegorz Pawelczak, Wayne Gaudin, Paul Garrett, Wei Liu, Richard Smedley-Stevenson, and David Beckingsale. 2017. Tealeaf: A mini-application to enable design-space explorations for iterative sparse linear solvers. In *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 842–849.
- [86] Richard C Murphy, Kyle B Wheeler, Brian W Barrett, and James A Ang. 2010. Introducing the graph 500. *Cray Users Group (CUG)* 19, 45–74 (2010), 22.
- [87] Seonjin Na, Geonhwa Jeong, Byung H Ahn, Aaron Jezghani, Jeffrey Young, Christopher J Hughes, Tushar Krishna, and Hyesoon Kim. 2025. Flexinfer: Flexible llm inference with cpu computations. *Proceedings of Machine Learning and Systems* 7 (2025).
- [88] Seonjin Na, Geonhwa Jeong, Byung Hoon Ahn, Jeffrey Young, Tushar Krishna, and Hyesoon Kim. 2024. Understanding performance implications of llm inference on cpus. In *2024 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 169–180.
- [89] Samuel Naffziger, Noah Beck, Thomas Burd, Kevin Lepak, Gabriel H Loh, Mahesh Subramony, and Sean White. 2021. Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 57–70.
- [90] Arash Nasr-Esfahany, Mohammad Alizadeh, Victor Lee, Hanna Alam, Brett W. Coon, David Culler, Vidushi Dadu, Martin Dixon, Henry M. Levy, Santosh Pandey, Parthasarathy Ranganathan, and Amir Yazdanbakhsh. 2025. Concorde: Fast and Accurate CPU Performance Modeling with Compositional Analytical-ML Fusion. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1480–1494.

- [91] Nevine Nassif, Ashley O. Munch, Carleton L. Molnar, Gerald Pasdast, Sitaraman V. Lyer, Zibing Yang, Oscar Mendoza, Mark Huddart, Srikrishnan Venkataraman, Sireesha Kandula, Rafi Marom, Alexandra M. Kern, Bill Bowhill, David R. Mulvihill, Srikanth Nimmagadda, Varma Kalidindi, Jonathan Krause, Mohammad M. Haq, Roopali Sharma, and Kevin Duda. 2022. Sapphire Rapids: The Next-Generation Intel Xeon Scalable Processor. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. 44–46. <https://doi.org/10.1109/ISSCC42614.2022.9731107>
- [92] Agustín Navarro-Torres, Jesús Alastruey-Benedé, Pablo Ibáñez-Marín, and Víctor Viñals-Yúfera. 2019. Memory hierarchy characterization of SPEC CPU2006 and SPEC CPU2017 on the Intel Xeon Skylake-SP. *Plos one* 14, 8 (2019), e0220135.
- [93] Nicholas Nethercote, Peter J Stuckey, Ralph Becket, Sebastian Brand, Gregory J Duck, and Guido Tack. 2007. MiniZinc: Towards a standard CP modelling language. In *International conference on principles and practice of constraint programming*. Springer, 529–543.
- [94] NVIDIA. 2026. NVIDIA Grace CPU Superchip Whitepaper. <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-cpu-superchip>.
- [95] SPEC org. 2026. SPEC CPU. <https://www.spec.org/cpu/>.
- [96] Biswabandan Panda. 2023. Clip: Load criticality based data prefetching for bandwidth-constrained many-core systems. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 714–727.
- [97] Reena Panda, Shuang Song, Joseph Dean, and Lizy K John. 2018. Wait of a decade: Did spec cpu 2017 broaden the performance horizon?. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 271–282.
- [98] Irma Esmer Papazian. 2020. New 3rd Gen Intel® Xeon® Scalable Processor (Codename: Ice Lake-SP). In *Hot Chips Symposium*. 1–22.
- [99] Gyeongseo Park, Minho Kim, Ki-Dong Kang, Yunhyeong Jeon, Seulki Kim, and Daehoon Kim. 2025. EcoCore: Dynamic Core Management for Improving Energy Efficiency in Latency-Critical Applications. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 1132–1146.
- [100] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [101] Tirthak Patel and Devesh Tiwari. 2020. Clite: Efficient and qos-aware co-location of multiple latency-critical jobs for warehouse scale computers. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 193–206.
- [102] Erez Perelman, Greg Hamerly, Michael Van Biesbrouck, Timothy Sherwood, and Brad Calder. 2003. Using simpoint for accurate and efficient simulation. *ACM SIGMETRICS Performance Evaluation Review* 31, 1 (2003), 318–319.
- [103] Aashish Phansalkar, Ajay Joshi, Lieven Eeckhout, and Lizy Kurian John. 2005. Measuring program similarity: Experiments with SPEC CPU benchmark suites. In *IEEE International Symposium on Performance Analysis of Systems and Software, 2005. ISPASS 2005*. IEEE, 10–20.
- [104] Aashish Phansalkar, Ajay Joshi, and Lizy K John. 2007. Analysis of redundancy and application balance in the SPEC CPU2006 benchmark suite. In *Proceedings of the 34th annual International Symposium on Computer architecture*. 412–423.
- [105] James C Phillips, Rosemary Braun, Wei Wang, James Gumbart, Emad Tajkhorshid, Elizabeth Villa, Christophe Chipot, Robert D Skeel, Laxmikant Kale, and Klaus Schulten. 2005. Scalable molecular dynamics with NAMD. *Journal of computational chemistry* 26, 16 (2005), 1781–1802.
- [106] Thomas Pohl, Markus Kowarschik, Jens Wilke, Klaus Iglberger, and Ulrich Rüde. 2003. Optimization and profiling of the cache performance of parallel lattice Boltzmann codes. *Parallel Processing Letters* 13, 04 (2003), 549–560.
- [107] CloverLeaf Mantevo Project. 2026. Compressible Euler equations on a Cartesian grid. <https://github.com/Mantevo/CloverLeaf>.
- [108] The Tukaani Project. 2026. XZ Utils. <https://github.com/tukaani-project/xz>.
- [109] Aleksandar Prokopec, Andrea Rosà, David Leopoldseider, Gilles Duboscq, Petr Tůma, Martin Studener, Lubomír Bulej, Yudi Zheng, Alex Villazón, Doug Simon, Thomas Würthinger, and Walter Binder. 2019. Renaissance: Benchmarking suite for parallel applications on the jvm. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 31–47.
- [110] Ritik Raj, Souvik Kundu, Ishita Vohra, Hong Wang, and Tushar Krishna. 2025. Towards Understanding, Analyzing, and Optimizing Agentic AI Execution: A CPU-Centric Perspective. *arXiv preprint arXiv:2511.00739* (2025).
- [111] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3505–3506.
- [112] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. Mlperf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 446–459.
- [113] Vijay Janapa Reddi, David Kanter, Peter Mattson, Jared Duke, Thai Nguyen, Ramesh Chukka, Ken Shiring, Koan-Sin Tan, Mark Charlebois, William Chou, Mostafa El-Khamy, Jungwook Hong, Tom St. John, Cindy Trinh, Michael Buch, Mark Mazumder, Relia Markovic, Thomas Atta, Fatih Cakir, Masoud Charkhabi, Xiaodong Chen, Cheng-Ming Chiang, Dave Dexter, Terry Heo, Gunther Schmuelling, Maryam Shabani, and Dylan Zika. 2022. MLPerf mobile inference benchmark: An industry-standard open-source machine learning benchmark for on-device AI. *Proceedings of Machine Learning and Systems* 4 (2022), 352–369.
- [114] George F Riley and Thomas R Henderson. 2010. The ns-3 network simulator. In *Modeling and tools for network simulation*. Springer, 15–34.
- [115] Tord Romstad, Marco Costalba, Joona Kiiski, and G Linscott. 2008. Stockfish: A strong open source chess engine. <https://stockfishchess.org>.
- [116] Norbert Schmitt, James Bucek, John Beckett, Aaron Cragin, Klaus-Dieter Lange, and Samuel Kounev. 2020. Performance, power, and energy-efficiency impact analysis of compiler optimizations on the spec cpu 2017 benchmark suite. In *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 292–301.
- [117] Norbert Schmitt, James Bucek, Klaus-Dieter Lange, and Samuel Kounev. 2020. Energy efficiency analysis of compiler optimizations on the spec cpu 2017 benchmark suite. In *Companion of the ACM/SPEC International Conference on Performance Engineering*. 38–41.
- [118] Yalong Shan, Yongkui Yang, Xuehai Qian, and Zhibin Yu. 2024. Guser: A GPGPU power stressmark generator. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1111–1124.
- [119] Alexander F Shchepetkin and James C McWilliams. 2005. The regional oceanic modeling system (ROMS): a split-explicit, free-surface, topography-following-coordinate oceanic model. *Ocean modelling* 9, 4 (2005), 347–404.
- [120] Yongming Shen, Michael Ferdman, and Peter Milder. 2017. Maximizing CNN accelerator efficiency through resource partitioning. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 535–547.
- [121] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. 2002. Automatically characterizing large scale program behavior. *ACM SIGPLAN Notices* 37, 10 (2002), 45–57.
- [122] Sarabjeet Singh and Manu Awasthi. 2019. Memory centric characterization and analysis of spec cpu2017 suite. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*. 285–292.
- [123] Teja Singh, Spence Oliver, Sundar Rangarajan, Shane Southard, Carson Henrion, Alex Schaefer, Brett Johnson, Sarah Bartaszewicz Tower, Kathy Hoover, Deepesh John, Ted Antoniadis, Shravan Lakshman, Vibhor Mittal, Brian Kasprzyk, Ross McCoy, Kurt Mohlman, Anitha Mohan, Hon-Hin Wong, Daryl Lieu, Russell Schreiber, Sahilpreet Singh, Nick Lance, Darryl Prudich, Justin Coppin, Tim Jackson, Anita Karegar, Ryan Miller, Sabeesh Balagangadharan, James Pistole, Wilson Li, and Michael McCabe. 2025. “Zen 5”: The AMD High-Performance 4nm x86-64 Microprocessor Core. In *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 68. 1–3. <https://doi.org/10.1109/ISSCC49661.2025.10904529>
- [124] Alan Smith, Gabriel H. Loh, Michael J. Schulte, Mike Ignatowski, Samuel Nafziger, Mike Mantor, Mark Fowler Nathan Kalyanasundharam, Vamsi Alla, Nicholas Malaya, Joseph L. Greathouse, Eric Chapman, and Raja Swaminathan. 2024. Realizing the AMD Exascale Heterogeneous Processor Vision: Industry Product. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 876–889.
- [125] Don Soltis and Stephen Robinson. 2023. The Next Generation of High Performance, Energy-Efficient Computing: Intel® Xeon® Processors Built on Efficient-Core.. In *HCS*. 1–16.
- [126] Kevin Song, Jiacheng Yang, Zixuan Wang, Jishen Zhao, Sihang Liu, and Gennady Pekhimenko. 2025. Hybridtier: an adaptive and lightweight cxl-memory tiering system. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 112–128.
- [127] Cloyce D Spradling. 2007. SPEC CPU2006 benchmark tools. *ACM SIGARCH Computer Architecture News* 35, 1 (2007), 130–134.
- [128] Ranjan Hebbar SR, Mounika Ponugoti, and Aleksandar Milenković. 2019. Battle of compilers: An experimental evaluation using spec cpu2017. In *2019 South-eastCon*. IEEE, 1–8.
- [129] Akshitha Sriraman, Abhishek Dhanotia, and Thomas F Wenisch. 2019. Softsku: Optimizing server architectures for microservice diversity@ scale. In *Proceedings of the 46th International Symposium on Computer Architecture*. 513–526.
- [130] Richard M Stallman. 2002. GNU compiler collection internals. *Free Software Foundation* 46 (2002), 765.

- [131] Jovan Stojkovic, Abraham Farrell, Gerasimos Gerogiannis, Zhangxiaowen Gong, C Hughes, and Josep Torrellas. 2026. Dorado: Clustered Hardware Cache Coherence for 1,000+ Cores. In *Proceedings of the 53rd Annual International Symposium on Computer Architecture (ISCA'26)*.
- [132] Jovan Stojkovic, Abraham Farrell, Zhangxiaowen Gong, Christopher J Hughes, and Josep Torrellas. 2026. AccelFlow: Orchestrating an On-Package Ensemble of Fine-Grained Accelerators for Microservices. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–17.
- [133] Jovan Stojkovic, Chunao Liu, Muhammad Shahbaz, and Josep Torrellas. 2023. μ Manycore: A Cloud-Native CPU for Tail at Scale. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [134] Wei Su, Abhishek Dhanotia, Carlos Torres, Jayneel Gandhi, Neha Gholkar, Shobhit Kanauiya, Maxim Naumov, Kalyan Subramanian, Valentin Andrei, Yifan Yuan, and Chunqiang Tang. 2025. DCPerf: An Open-Source, Battle-Tested Performance Benchmark Suite for Datacenter Workloads. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1717–1730.
- [135] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyi Xu, and Nam Sung Kim. 2023. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 105–121.
- [136] John R Tramm, Andrew R Siegel, Benoit Forget, and Colin Josey. 2014. Performance analysis of a reduced data movement algorithm for neutron cross section data in monte carlo simulations. In *International Conference on Exascale Applications and Software*. Springer, 39–56.
- [137] Arya Tschand, Arun Tejvase Raghunath Rajan, Sachin Idgunji, Anirban Ghosh, Jeremy Holleman, Csaba Kiraly, Pawan Ambalkar, Ritika Borkar, Ramesh Chukka, Trevor Cockrell, Oliver Curtis, Grigori Fursin, Miro Hodak, Hiwot Kassa, Anton Lokhmotov, Dejan Miskovic, Yuechao Pan, Manu Prasad Manmathan, Liz Raymond, Tom St. John, Arjun Suresh, Rowan Taubitz, Sean Zhan, Scott Wasson, David Kanter, and Vijay Janapa Reddi. 2025. MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from μ Watts to MWatts for Sustainable AI. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1201–1216.
- [138] Taegeon Um, Byungsoo Oh, Byeongchan Seo, Minhyeok Kweon, Goeun Kim, and Woo-Yeon Lee. 2023. Fastflow: Accelerating deep learning model training with smart offloading of input data pipeline. *Proceedings of the VLDB Endowment* 16, 5 (2023), 1086–1099.
- [139] Guido Van Rossum. 1995. *Python reference manual*. Vol. 111. Centrum voor Wiskunde en Informatica Amsterdam.
- [140] András Varga and Rudolf Hornig. 2008. An overview of the OMNet++ simulation environment. In *Proceedings of the 1st international conference on Simulation tools and techniques for communications, networks and systems & workshops*. 1–10.
- [141] Markus Velten, Robert Schöne, Thomas Ilsche, and Daniel Hackenberg. 2022. Memory performance of AMD EPYC Rome and Intel Cascade Lake SP server processors. In *Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering*. 165–175.
- [142] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. 2024. Freshllms: Refreshing large language models with search engine augmentation. In *Findings of the Association for Computational Linguistics: ACL 2024*. 13697–13720.
- [143] Doug Walker, Michael Dolan, and Patrick Hodoul. 2020. The ASWF takes OpenColorIO to the next level. In *Proceedings of the 2020 Digital Production Symposium*. 1–10.
- [144] Siqi Wang, Tianyu Feng, Hailong Yang, Xin You, Bangduo Chen, Tongxuan Liu, Zhongzhi Luan, and Depei Qian. 2024. AtRec: Accelerating recommendation model training on CPUs. *IEEE Transactions on Parallel and Distributed Systems* 35, 6 (2024), 905–918.
- [145] Chris Welty. 2026. NTest othello program. <https://github.com/weltyc/ntest>.
- [146] Zheng Xu, Dehao Kong, Jiaxin Liu, Jinxi Li, Jingxiang Hou, Xu Dai, Chao Li, Shaojun Wei, Yang Hu, and Shouyi Yin. 2025. WSC-LLM: Efficient LLM Service and Architecture Co-exploration for Wafer-scale Chips. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1–17.
- [147] Jawad Haj Yahya, Haris Volos, Davide B. Bartolini, Georgia Antoniou, Jeremie S. Kim, Zhe Wang, Kleovoulos Kalaitzidis, Tom Rollet, Zhirui Chen, Ye Geng, Onur Mutlu, and Yiannakis Sazeides. 2022. Agilewatts: An energy-efficient cpu core idle-state architecture for latency-sensitive server applications. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 835–850.
- [148] Dong Ye, Joydeep Ray, Christophe Harle, and David Kaeli. 2006. Performance characterization of SPEC CPU2006 integer benchmarks on x86-64 architecture. In *2006 IEEE International Symposium on Workload Characterization*. IEEE, 120–127.
- [149] Chengye Yu, Tianyu Wang, Zili Shao, Linjie Zhu, Xu Zhou, and Song Jiang. 2024. Twinpilots: A new computing paradigm for gpu-cpu parallel llm inference. In *Proceedings of the 17th ACM International Systems and Storage Conference*. 91–103.
- [150] Mohamed Zahran. 2019. *Heterogeneous computing: Hardware and software perspectives*. Morgan & Claypool.
- [151] Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. 2015. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In *Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays*. 161–170.
- [152] Shiqing Zhang, Mahmood Naderan-Tahan, Magnus Jahre, and Lieven Eeckhout. 2023. Characterizing multi-chip GPU data sharing. *ACM Transactions on Architecture and Code Optimization* 20, 4 (2023), 1–24.
- [153] Xianyi Zhang, Martin Kroeker, Werner Saar, Wang Qian, Zaheer Chothia, Shaohu Chen, and Wen Luo. 2026. OpenBLAS: An Optimized BLAS Library. <https://www.openblas.net>.
- [154] Yuxuan Zhang, Tanvir Ahmed Khan, Gilles Pokam, Baris Kasikci, Heiner Litz, and Joseph Devietti. 2022. Ocolos: Online code layout optimizations. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 530–545.
- [155] Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jishi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, Wenfeng Liang, Ying He, Yuqing Wang, Yuxuan Liu, and Y.X. Wei. 2025. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1731–1745.
- [156] Kaiyang Zhao, Kaiwen Xue, Ziqi Wang, Dan Schatzberg, Leon Yang, Antonis Manousis, Johannes Weiner, Rik Van Riel, Bikash Sharma, Chunqiang Tang, and Dimitrios Skarlatos. 2023. Contiguitas: The pursuit of physical memory contiguity in datacenters. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [157] Laiping Zhao, Yanan Yang, Yiming Li, Xian Zhou, and Keqiu Li. 2021. Understanding, predicting and scheduling serverless workloads under partial interference. In *Proceedings of the International conference for high performance computing, networking, storage and analysis*. 1–15.
- [158] Mark Zhao, Niket Agarwal, Aarti Basant, Bugra Gedik, Satadru Pan, Mustafa Ozdal, Rakesh Komuravelli, Jerry Pan, Tianshu Bao, Haowei Lu, Sundaram Narayanan, Jack Langman, Kevin Wilfong, Harsha Rastogi, Carole-Jean Wu, Christos Kozyrakis, and Parik Pol. 2022. Understanding data storage and ingestion for large-scale deep recommendation model training: Industrial product. In *Proceedings of the 49th annual international symposium on computer architecture*. 1042–1057.
- [159] Yufeng Zhou, Alan L Cox, Sandhya Dwarkadas, and Xiaowan Dong. 2023. The Impact of Page Size and Microarchitecture on Instruction Address Translation Overhead. *ACM Transactions on Architecture and Code Optimization* 20, 3 (2023), 1–25.
- [160] Zhuangzhuang Zhou, Vaibhav Gogte, Nilay Vaish, Chris Kennelly, Patrick Xia, Svilen Kanev, Tipp Moseley, Christina Delimitrou, and Parthasarathy Ranganathan. 2024. Characterizing a memory allocator at warehouse scale. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 192–206.
- [161] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Table 9: Dynamic instruction count (billions), application domain, instruction mix, and IPC for each SPEC CPU26 benchmark measured on a representative Intel Sapphire Rapids Platinum platform (CPU-C in Table 1).

Benchmark	Domain	Icount	Loads	Stores	Branches	IPC
<i>SPEC CPU26 Integer Rate – 14 workloads</i>						
706.stockfish_r	Games (Chess) [115]	6507	22.0	9.9	10.4	3.625
707.ntest_r	Games (Othello) [145]	2507	25.0	9.6	9.2	3.268
708.sqlite_r	Database [37]	1716	26.9	11.9	20.9	2.228
710.omnetpp_r	Network Sim. [140]	1583	31.9	17.5	20.5	2.103
714.cpython_r	Lang. Runtime [139]	1475	27.9	15.8	21.4	2.843
721.gcc_r	Compiler [130]	1503	28.0	11.3	21.8	0.551
723.llvm_r	Compiler [72]	1534	26.0	13.7	20.8	1.484
727.cppcheck_r	Static Analysis [82]	1286	22.5	9.6	26.7	2.228
729.abc_r	EDA [17]	1400	26.2	8.9	16.7	2.187
734.vpr_r	EDA (FPGA) [12]	1367	30.9	11.2	19.2	2.097
735.gem5_r	Arch. Sim. [15]	1659	30.2	14.7	20.9	2.068
750.sealcrypto_r	Cryptography [21]	3087	12.0	4.7	1.9	4.961
753.ns3_r	Network Sim. [114]	1432	29.4	16.8	22.2	2.230
777.zstd_r	Compression [33]	1817	22.2	9.0	13.3	1.911
<i>SPEC CPU26 Integer Speed – 13 workloads</i>						
801.xz_s	Compression [108]	17757	22.2	7.4	14.4	1.008
807.ntest_s	Games (Othello) [145]	151005	20.8	7.8	6.7	3.460
817.flac_s	Compression	90970	17.6	2.3	4.4	4.156
821.gcc_s	Compiler [130]	109486	26.8	12.5	21.7	2.016
823.llvm_s	Compiler [72]	103105	22.0	11.8	23.1	1.896
827.cppcheck_s	Static Analysis [82]	90423	23.0	11.2	26.5	2.375
829.abc_s	EDA [17]	1433	25.1	11.7	18.1	0.858
834.vpr_s	EDA (FPGA) [12]	3117	30.7	11.1	19.4	1.863
835.gem5_s	Arch. Sim. [15]	2858	29.4	13.5	17.6	1.805
838.diamond_s	Bioinformatics [19]	146966	20.1	6.8	5.5	3.203
846.minizinc_s	CSP Solver [93]	5062	26.5	18.2	15.9	1.228
853.ns3_s	Network Sim. [114]	11053	28.9	14.3	21.0	1.662
854.graph500_s	Graph Analytics [86]	37168	36.2	0.9	25.7	1.539
<i>SPEC CPU26 Floating Point Rate – 12 workloads</i>						
709.cactus_r	Physics (GR) [42]	1456	51.9	7.9	1.1	1.696
722.palm_r	Climate [83]	3272	39.0	9.1	5.0	3.187
731.astcenc_r	Img. Compress. [7]	2615	28.3	6.5	8.7	2.718
736.ocio_r	Image Process. [143]	2484	24.2	7.5	9.8	3.269
737.gmsh_r	Mesh Generation [39]	1086	29.2	12.1	17.2	1.585
748.flightdm_r	Aerospace [11]	1721	29.5	14.2	18.8	3.071
749.fotonik3d_r	Photonics [5]	1291	36.8	13.7	1.8	0.785
765.roms_r	Ocean Modeling [119]	2738	34.8	8.4	7.3	1.830
766.femflow_r	Fluid Dynamics [69]	5012	34.9	20.1	6.9	3.265
767.nest_r	Neuroscience [40]	1848	33.5	12.3	14.0	2.844
772.marian_r	NLP [64]	6389	8.7	1.3	3.0	3.953
782.lbm_r	Fluid Dynamics [136]	2236	21.2	10.9	0.7	1.241
<i>SPEC CPU26 Floating Point Speed – 13 workloads</i>						
800.pot3d_s	Astrophysics [30]	7603	34.8	8.1	9.5	0.754
803.sph_exa_s	HPC (SPH) [47]	64626	24.8	3.4	11.2	2.465
809.cactus_s	Physics (GR) [42]	29190	51.9	8.1	1.6	1.338
811.tealeaf_s	Physics (Heat) [85]	40570	20.2	4.9	8.8	1.617
816.nab_s	Mol. Dynamics [79]	67717	31.2	5.7	11.9	2.441
820.cloverleaf_s	Physics (Hydro) [107]	25781	33.2	4.7	5.9	1.349
822.palm_s	Climate [83]	48883	38.2	8.9	6.4	1.920
849.fotonik3d_s	Photonics [5]	17777	56.0	9.8	2.7	0.955
857.namd_s	Mol. Dynamics [105]	168881	26.4	6.6	2.2	3.929
865.roms_s	Ocean Modeling [119]	28484	34.7	8.6	7.9	1.574
867.nest_s	Neuroscience [40]	66774	30.1	9.3	14.7	1.790
872.marian_s	NLP [64]	65980	10.8	2.8	3.8	3.258
881.neutron_s	Nuclear Physics [106]	33545	25.8	11.3	9.1	1.204

Table 10: Detailed per-workload RSS of SPEC CPU17 (appendix for Figure 5a), in GB.

Workload	C_glibc	C_mi_no_thp	C_mi_thp	C_tc_no_thp	C_tc_thp	I_glibc	I_mi_no_thp	I_mi_thp	I_tc_no_thp	I_tc_thp
500.perlbench_r	0.54	0.57	0.58	0.57	0.57	0.56	0.59	0.59	0.60	0.59
502.gcc_r	1.66	1.63	1.64	1.68	1.67	1.67	1.66	1.64	1.70	1.69
503.bwaves_r	1.15	1.15	1.18	1.16	1.16	1.16	1.16	1.16	1.17	1.17
505.mcf_r	0.94	0.94	0.95	1.23	1.23	0.96	0.96	0.96	1.24	1.24
507.cactuBSSN_r	1.12	1.12	1.20	1.12	1.12	1.14	1.14	1.14	1.15	1.15
508.namd_r	0.51	0.50	0.56	0.51	0.51	0.53	0.54	0.54	0.55	0.55
510.parest_r	0.75	0.77	0.80	0.81	0.81	0.77	0.81	0.81	0.84	0.85
511.povray_r	0.45	0.44	0.45	0.45	0.45	0.42	0.43	0.46	0.46	0.48
519.lbm_r	0.75	0.75	0.75	0.75	0.75	0.76	0.76	0.76	0.77	0.77
520.omnetpp_r	0.58	0.58	0.58	0.58	0.59	0.60	0.59	0.59	0.60	0.60
521.wrf_r	0.54	0.55	0.61	0.54	0.55	0.56	0.58	0.58	0.58	0.58
523.xalancbmk_r	0.81	0.81	0.82	0.84	0.84	0.83	0.83	0.85	0.86	0.86
525.x264_r	0.50	0.50	0.56	0.51	0.51	0.52	0.53	0.53	0.53	0.53
526.blender_r	0.96	1.08	1.10	1.01	1.01	0.98	1.12	1.12	1.05	1.05
527.cam4_r	1.22	1.24	1.31	1.25	1.25	1.23	1.26	1.26	1.27	1.27
531.deepsjeng_r	1.03	1.03	1.03	1.04	1.04	1.04	1.04	1.04	1.05	1.05
538.imagick_r	0.63	0.72	0.72	0.72	0.72	0.64	0.73	0.73	0.74	0.74
541.leela_r	0.40	0.40	0.45	0.45	0.40	0.41	0.42	0.47	0.42	0.42
544.nab_r	0.49	0.53	0.54	0.51	0.52	0.51	0.55	0.55	0.52	0.52
548.exchange2_r	0.40	0.46	0.46	0.45	0.44	0.47	0.41	0.47	0.42	0.46
549.fotonik3d_r	1.17	1.18	1.20	1.18	1.18	1.19	1.19	1.19	1.20	1.20
554.roms_r	1.17	1.17	1.23	1.18	1.18	1.18	1.19	1.19	1.20	1.20
557.xz_r	1.11	1.16	1.17	1.10	1.28	1.11	1.16	1.16	1.20	1.29

Table 11: Detailed per-workload RSS of SPEC CPU26 (appendix for Figure 5a), in GB.

Workload	C_glibc	C_mi_no_thp	C_mi_thp	C_tc_no_thp	C_tc_thp	I_glibc	I_mi_no_thp	I_mi_thp	I_tc_no_thp	I_tc_thp
706.stockfish_r	2.06	2.06	2.14	2.13	2.13	2.08	2.15	2.15	2.15	2.15
707.ntest_r	0.64	0.63	0.64	0.64	0.64	0.66	0.67	0.67	0.67	0.66
708.sqlite_r	1.59	1.59	1.43	1.35	1.35	1.61	1.43	1.45	1.35	1.35
709.cactus_r	2.05	2.05	2.17	2.05	2.05	2.08	2.09	2.09	2.09	2.09
710.omnetpp_r	1.08	1.15	1.07	1.13	1.09	1.16	1.14	1.15	1.10	1.08
714.cpython_r	1.53	1.51	1.57	1.69	1.71	1.53	1.57	1.58	1.69	1.70
721.gcc_r	1.34	1.34	1.33	1.72	1.72	1.38	1.36	1.35	1.76	1.75
722.palm_r	2.13	2.15	2.19	2.18	2.19	2.14	2.19	2.19	2.21	2.21
723.llvm_r	1.76	1.76	1.69	1.72	1.72	1.79	1.70	1.70	1.75	1.75
727.cppcheck_r	0.97	0.97	0.95	0.96	0.96	0.99	0.97	0.97	0.98	0.98
729.abc_r	1.84	1.85	1.86	1.86	1.87	1.87	1.86	1.86	1.89	1.89
731.astcenc_r	0.65	0.64	0.64	0.64	0.64	0.67	0.67	0.67	0.67	0.67
734.vpr_r	1.98	2.00	2.06	1.80	1.81	2.03	2.02	2.12	1.85	1.85
735.gem5_r	1.13	1.10	1.06	1.05	1.10	1.16	1.07	1.07	1.08	1.08
736.ocio_r	1.92	1.92	1.93	1.92	1.93	1.94	1.94	1.94	1.95	1.95
737.gmsh_r	1.39	1.39	1.47	1.45	1.45	1.42	1.49	1.56	1.52	1.52
748.flightdm_r	0.64	0.65	0.65	0.64	0.64	0.67	0.67	0.67	0.67	0.67
749.fotonik3d_r	2.11	2.11	2.15	2.12	2.12	2.13	2.14	2.14	2.15	2.15
750.sealcrypto_r	0.93	0.93	0.96	0.93	0.95	0.96	0.96	0.96	0.96	0.96
753.ns3_r	0.64	0.64	0.64	0.64	0.64	0.67	0.67	0.67	0.67	0.66
760.rocksdb_r	0.77	0.77	0.75	0.76	0.76	0.79	0.77	0.77	0.78	0.78
765.roms_r	1.56	1.56	1.62	1.57	1.58	1.59	1.60	1.60	1.62	1.62
766.femflow_r	0.73	0.71	0.69	0.71	0.72	0.75	0.72	0.72	0.75	0.75
767.nest_r	1.46	1.46	1.72	1.48	1.48	1.49	1.74	1.74	1.50	1.50
772.marian_r	1.44	1.44	1.42	1.39	1.39	1.46	1.46	1.46	1.39	1.39
777.zstd_r	0.84	0.84	0.84	0.84	0.84	0.86	0.86	0.86	0.86	0.86
782.lbm_r	2.03	2.03	2.03	2.03	2.04	2.05	2.05	2.05	2.06	2.06

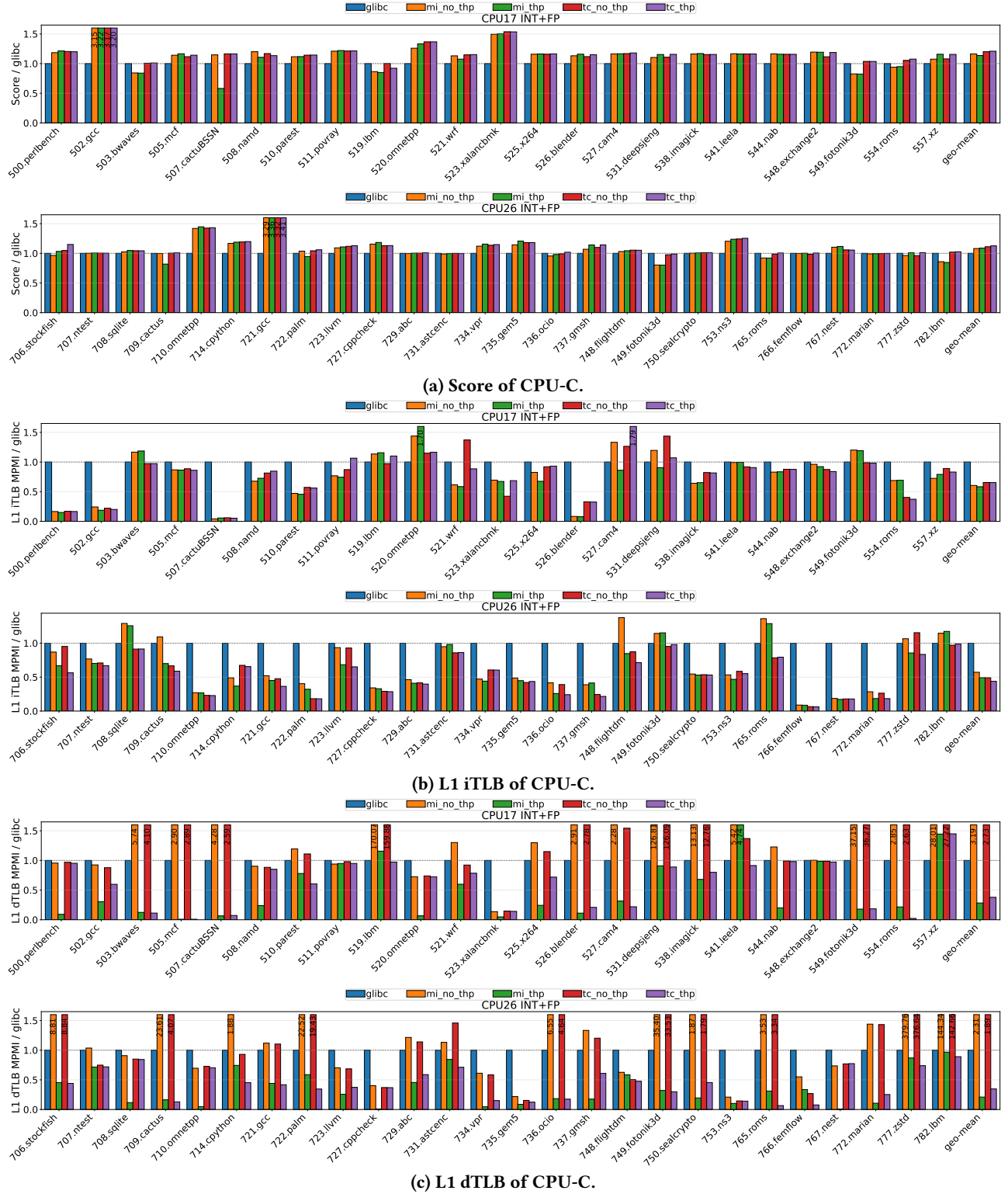


Figure 15: Detailed per-workload performance of different allocators on SPEC CPU17 and SPEC CPU26 (Figure 6, CPU-C).

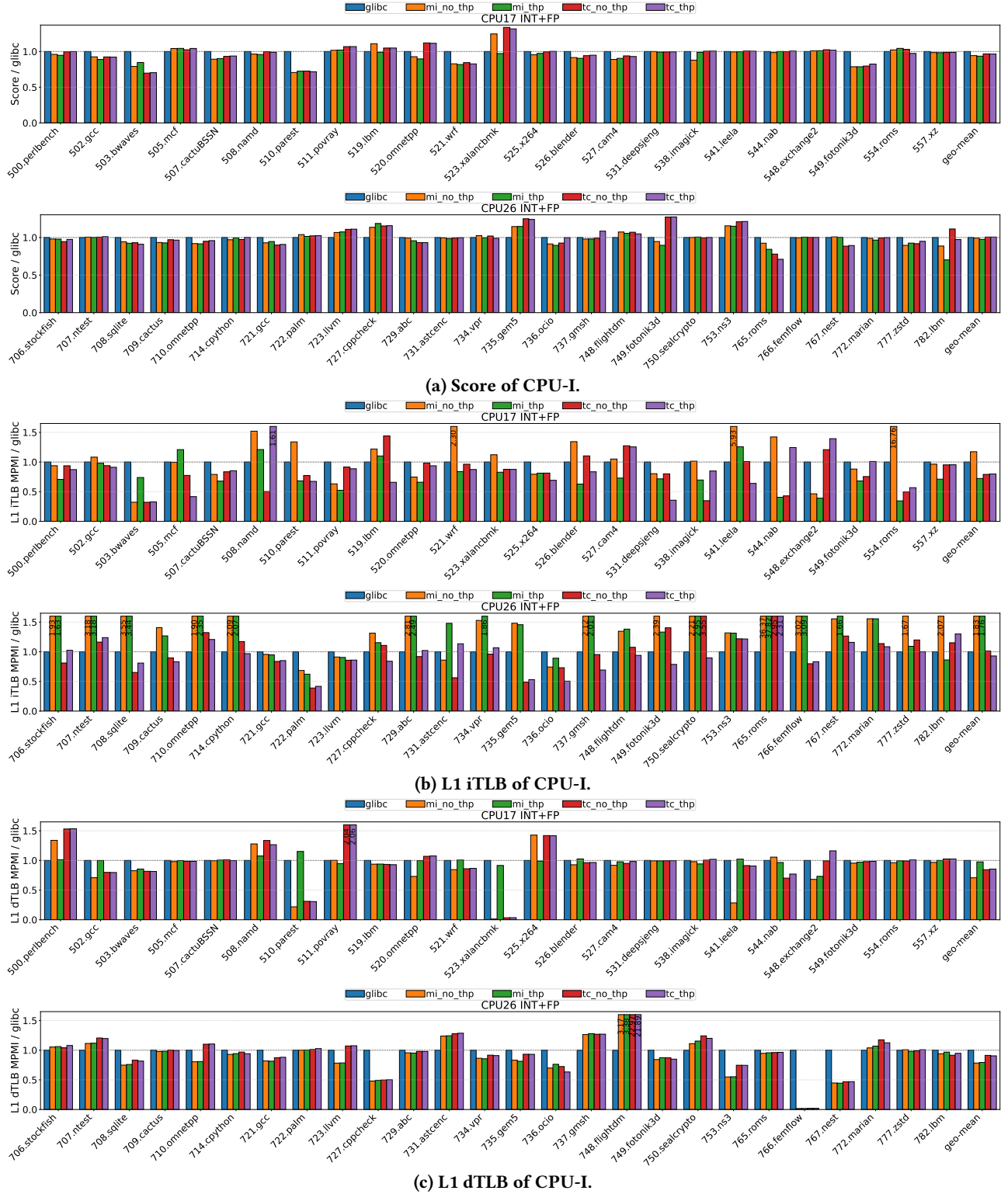
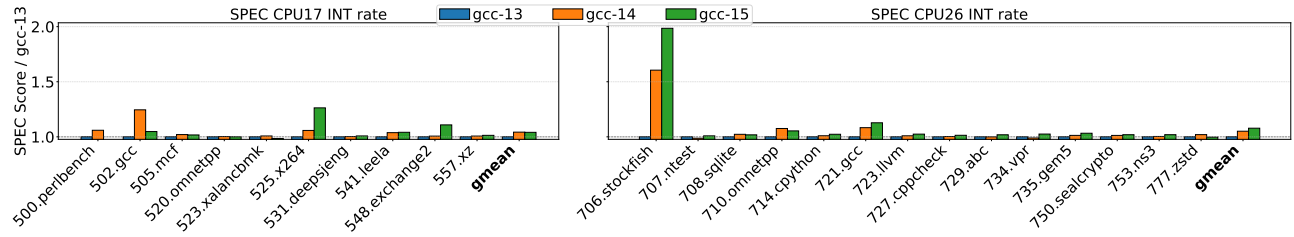
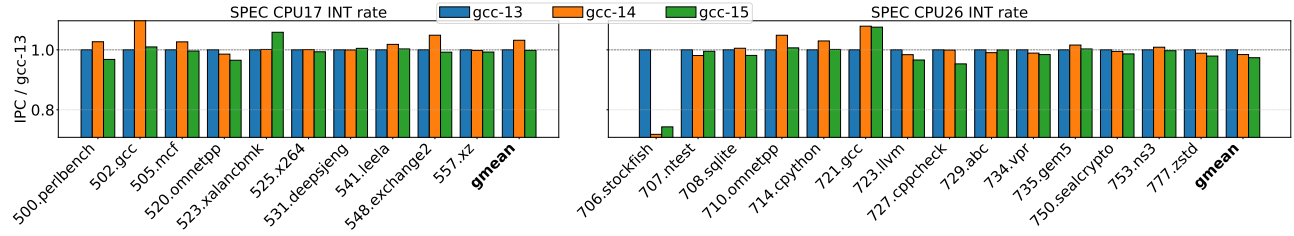


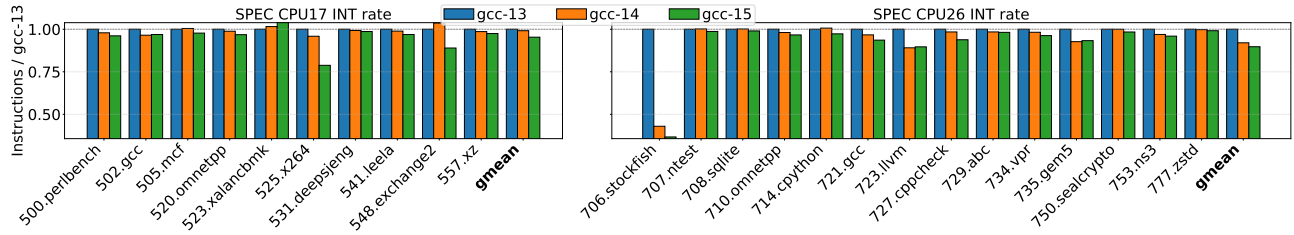
Figure 16: Detailed per-workload performance of different allocators on *SPEC CPU17* and *SPEC CPU26* (Figure 6, CPU-I).



(a) Score of INT Rate workloads.

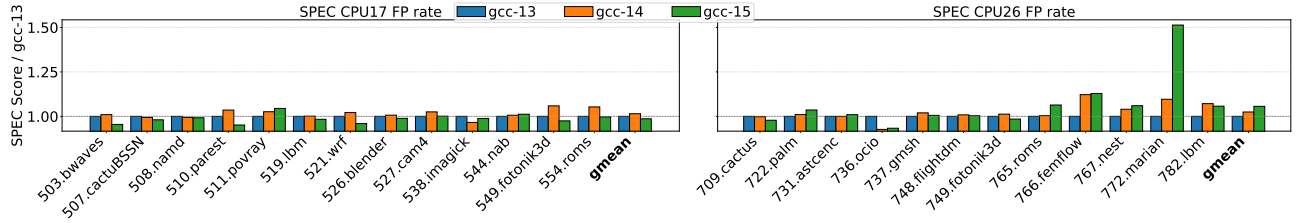


(b) IPC of INT Rate workloads.

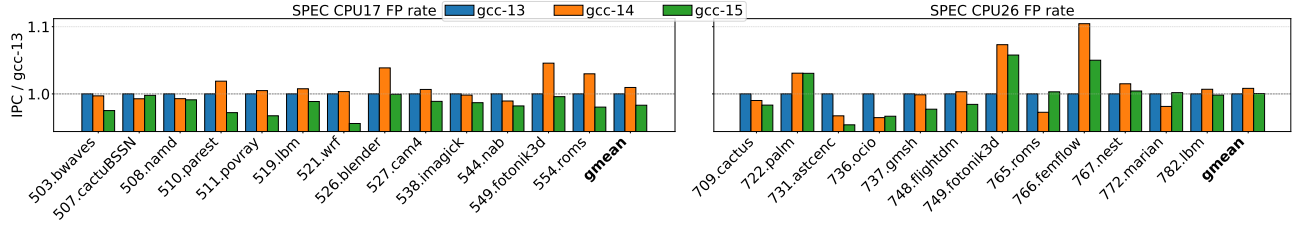


(c) Instruction counts of INT Rate workloads.

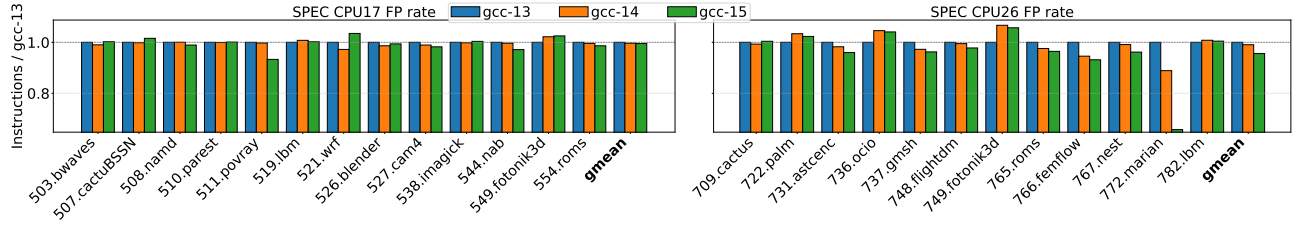
Figure 17: Detailed per-workload performance of different compilers on *SPEC CPU17* and *SPEC CPU26* INT Rate workloads (Figure 8).



(a) Score of FP Rate workloads.



(b) IPC of FP Rate workloads.



(c) Instruction counts of FP Rate workloads.

Figure 18: Detailed per-workload performance of different compilers on *SPEC CPU17* and *SPEC CPU26* FP Rate workloads (Figure 8).

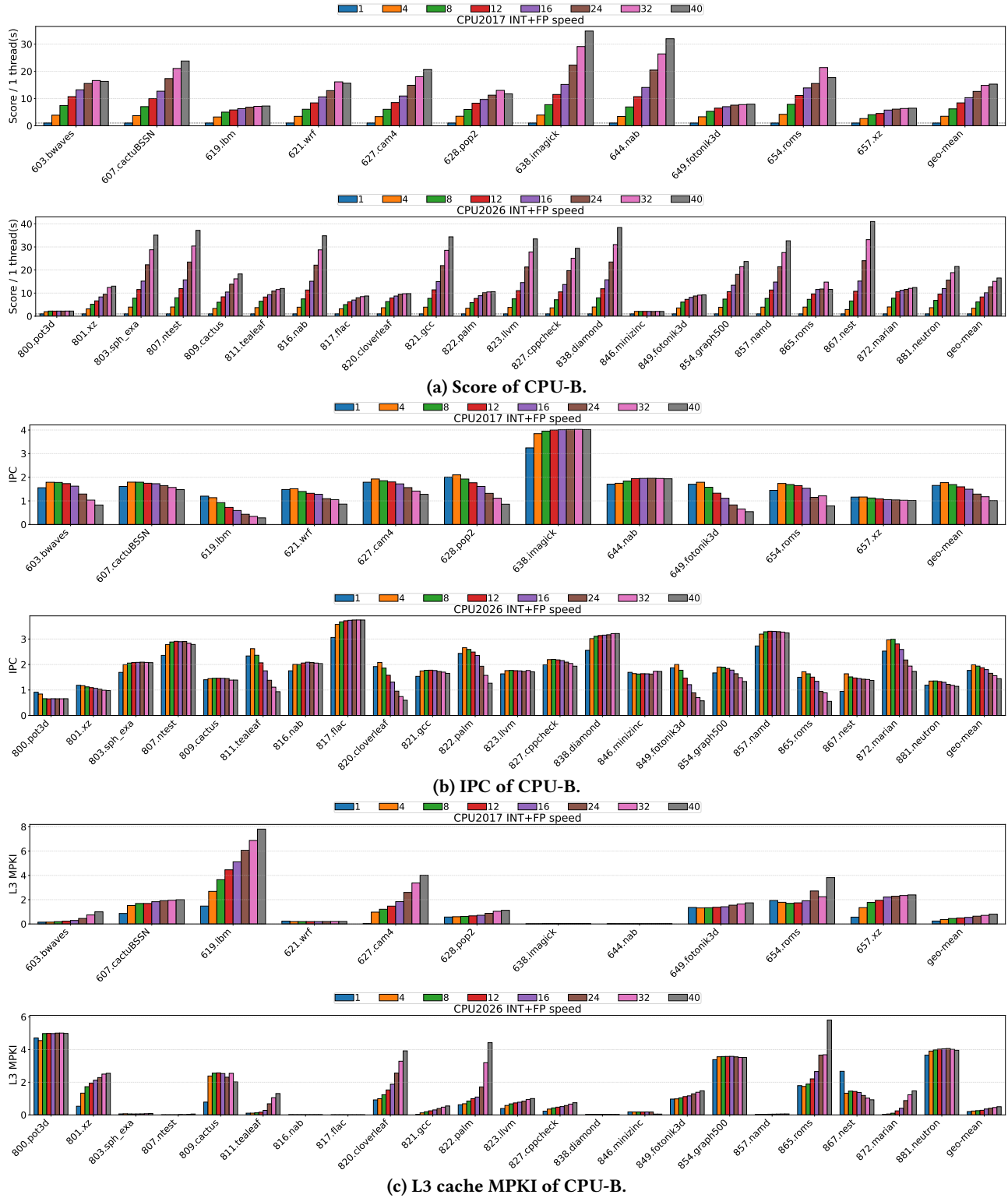


Figure 19: Detailed per-workload performance of Speed workload scaling on *SPEC CPU17* and *SPEC CPU26* (Figure 10, CPU-B).

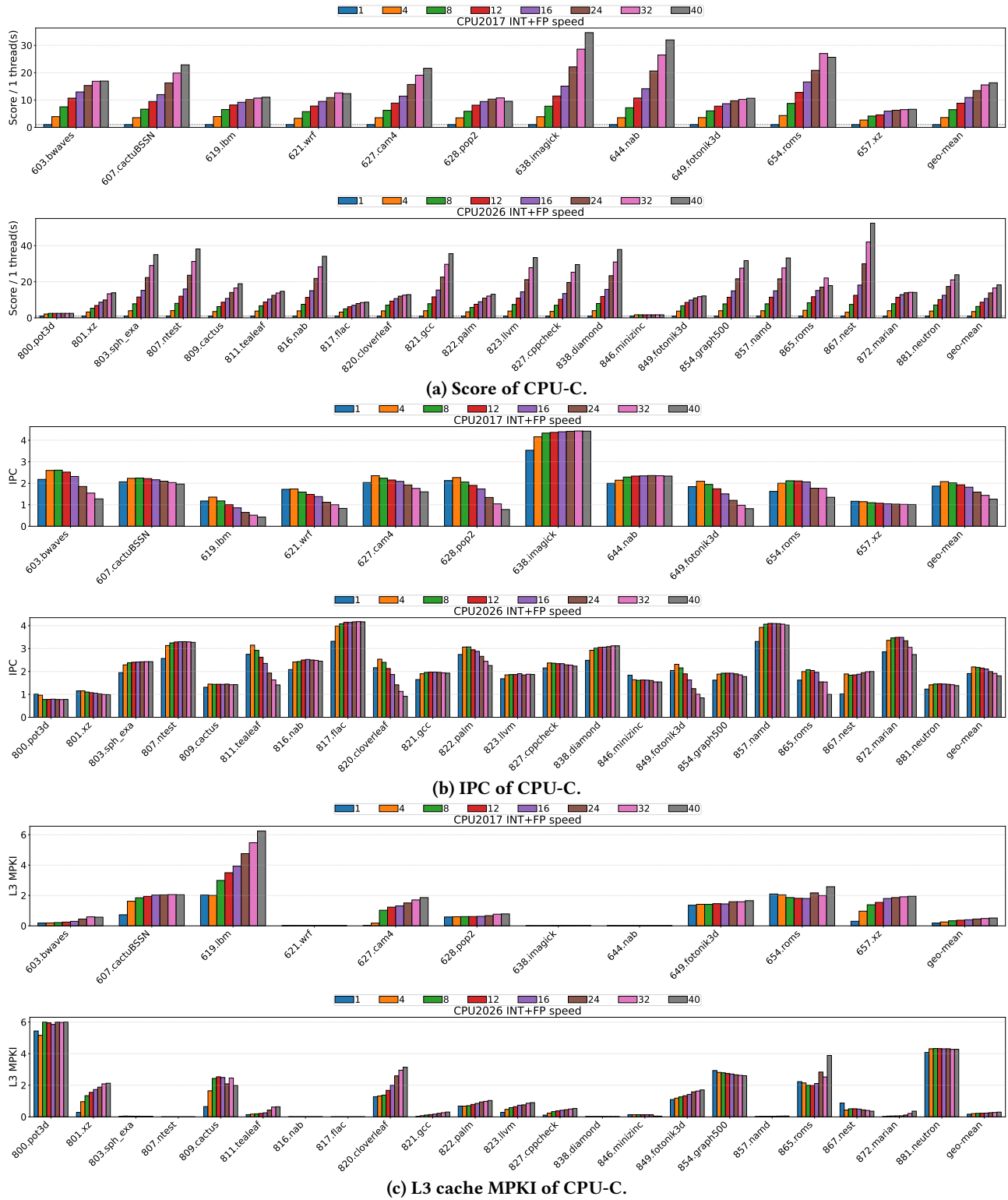


Figure 20: Detailed per-workload performance of Speed workload scaling on *SPEC CPU17* and *SPEC CPU26* (Figure 10, CPU-C).